Virtualizer Studio - User Guide

Contents

1. Preface
   1.1. About This Manual
   1.2. Documentation Conventions
   1.3. Terminology
   1.4. Customer Support
   1.5. Synopsys Statement on Inclusivity and Diversity
2. Chapter 1 Use Cases for Virtualizer Studio
   2.1. 1.1 TLM Creation
   2.2. 1.2 VDK Creation
   2.3. 1.3 VDK Debug
   2.4. 1.4 VDK Creator Terminology
      2.4.1. 1.4.1 Virtualizer Development Kit (VDK)
      2.4.2. 1.4.2 VDK Project
      2.4.3. 1.4.3 VDK Building Block
      2.4.4. 1.4.4 Fixed VDK
      2.4.5. 1.4.5 Extensible VDK
      2.4.6. 1.4.6 Example VDK
      2.4.7. 1.4.7 Multi-Node VDK
      2.4.8. 1.4.8 VDK Node
      2.4.9. 1.4.9 Multi-Node VDK Project
      2.4.10. 1.4.10 VP Config
      2.4.11. 1.4.11 TLM Model
      2.4.12. 1.4.12 TLM Project
      2.4.13. 1.4.13 Extensible TLM
      2.4.14. 1.4.14 TLM Building Block
3. Chapter 2 Getting Started with Virtualizer Studio
   3.1. 2.1 Configuring the Virtualizer Environment
   3.2. 2.2 Launching the Virtualizer Studio IDE
   3.3. 2.3 Selecting the Eclipse Workspace
   3.4. 2.4 Virtualizer Studio Welcome Screen
   3.5. 2.5 Example Browser
   3.6. 2.6 GUI Icons
   3.7. 2.7 Launching the Virtualizer Studio Shell Application
   3.8. 2.8 Command-Line Arguments
4. Chapter 3 Python Console
   4.1. 3.1 Running Scripts
   4.2. 3.2 Context-Sensitive Help
   4.3. 3.3 Python Objects
   4.4. 3.4 APIs
   4.5. 3.5 Console View
      4.5.1. 3.5.1 Creating Multiple Console Tabs
      4.5.2. 3.5.2 Available Consoles
   4.6. 3.6 Migrating Scripts to Become Compatible with Python 3
      4.6.1. 3.6.1 Using Futurize to Upgrade Existing Scripts
      4.6.2. 3.6.2 Additional References
5. Chapter 4 IP and VDK Packaging Flows
   5.1. 4.1 Overview
      5.1.1. 4.1.1 Library Management
      5.1.2. 4.1.2 IP Library Types
      5.1.3. 4.1.3 Supply Chain Example
      5.1.4. 4.1.4 Build System Integration

5.2. 4.2 TLM Building Blocks
5.2.1. 4.2.1 Library Packaging
5.2.2. 4.2.2 Test Packaging
5.3. 4.3 TLM Models
5.3.1. 4.3.1 Library Packaging
5.3.2. 4.3.2 Extensible Packaging
5.3.3. 4.3.3 Test Packaging
5.4. 4.4 VDK Building Blocks
5.4.1. 4.4.1 Library Packaging
5.4.2. 4.4.2 Extensible Packaging
5.4.3. 4.4.3 IP Library Packaging
5.5. 4.5 Virtualizer Development Kits
5.5.1. 4.5.1 Fixed VDK Packaging
5.5.2. 4.5.2 Extensible VDK Packaging
5.5.3. 4.5.3 No VDK Packaging
5.6. 4.6 Fixed VDKs
5.6.1. 4.6.1 Fixed VDK Re-Packaging
6. Chapter 5 Virtualizer Studio Build System Integration
6.1. 5.1 Supported Projects and Actions
6.2. 5.2 Usage
6.2.1. 5.2.1 Creating Packages
6.2.2. 5.2.2 Running Tests
6.3. 5.3 Licensing Requirements
6.4. 5.4 Quick References
6.4.1. 5.4.1 TLM Building Block Projects
6.4.2. 5.4.2 TLM Model Projects
6.4.3. 5.4.3 VDK Building Block Projects
6.4.4. 5.4.4 VDK Projects
6.4.5. 5.4.5 Fixed VDK Projects
6.4.6. 5.4.6 Python-Based Projects
7. Chapter 6 Virtualizer Studio Building Block Compliance Testing
7.1. 6.1 Test Scenarios
7.1.1. 6.1.1 Run a VDK Containing the Building Block
7.1.2. 6.1.2 Package a VDK Containing the Building Block
7.1.3. 6.1.3 Creating a Subsystem Containing the Building Block
7.1.4. 6.1.4 Runtime Disabling the Building Block
7.1.5. 6.1.5 Presence of Collateral Files in Building Block Library
7.2. 6.2 Running the Tests
7.2.1. 6.2.1 Command-Line Arguments
7.2.2. 6.2.2 Running Multiple Tests
7.3. 6.3 Test Configuration Options
7.3.1. 6.3.1 Fallback Template Arguments
7.3.2. 6.3.2 Skipping Tests
7.3.3. 6.3.3 Parameter Overrides
7.3.4. 6.3.4 Additional Libraries
7.4. 6.4 Test Results
8. Appendix A Enhanced Security for Sensitive IP and VDKs
8.1. A.1 Provide Maximum Value to End Users
8.2. A.2 Protect Sensitive Areas of a Design
8.3. A.3 Protect Sensitive IP Inside Individual Models
8.4. A.4 Details of Restrictions for Each User

< >

1. Preface

The preface of the *Virtualizer Studio User Guide* describes:

- About This Manual
- Documentation Conventions
- Terminology
- Customer Support
- Synopsys Statement on Inclusivity and Diversity

< > < >

1.1. About This Manual

Virtualizer Studio is a tool shipped with Virtualizer, which provides you with a single development environment in which you can carry out the development, debugging and running of both TLM models and VDKs.

Virtualizer Studio is provided as an Eclipse-based IDE, meaning that it comprises of a variety of application-specific features, and also leverages standard Eclipse capabilities and features in a way which is consistent with other Eclipse-based IDEs.

This manual is organized as follows:

- Use Cases for Virtualizer Studio describes the primary use cases of Virtualizer Studio.
- Getting Started with Virtualizer Studio explains how to use Virtualizer Studio.
- Python Console describes the Python console, which allows you to interact with the different underlying technologies via a command-line interface.
- IP and VDK Packaging Flows describes various flows of building and distributing IP libraries and VDKs supported in Virtualizer Studio.
- Virtualizer Studio Build System Integration describes the command-line tool that provides the ability to integrate into a build and regression system to generate and test VDK and TLM projects.
- Virtualizer Studio Building Block Compliance Testing explains the building block compliance testing in Virtualizer Studio.
- Enhanced Security for Sensitive IP and VDKs describes the enhanced security and restrictions for sensitve IP and VDKs.

< > < >

1.2. Documentation Conventions

This section lists and explains the documentation conventions used throughout this manual.

Convention	Description and Examples
italic	Is used in running text for: • GUI elements. For example: The *Enumeration* field contains a space-separated list of values. • New terms. For example: A *protocol library* is a collection of protocol definitions. • Web sites. For example: For more information, see *www.eclipse.org*. • E-mail addresses. For example: Contact customer support via e-mail at *vp_support@synopsys.com*. • Manual names. For example: The preface of the *Analysis Manual* describes:
`courier`	Is used for: • Code text. For example: `list_library_configurations myConfig` In this example, `myConfig` is used. • System messages. For example:

	`JVM not found.` • Text you must type literally. For example: At the prompt, type `go`. • Names (of environment variables, commands, utilities, prompts, paths, macros, and so on). For example: The `build-options` command sets build parameters.
courier italic	Indicates variables. For example: *`scope`* specifies a module, a channel, or a refined port.
bold	Serves to draw your attention to the text in question. For example: `coreId = cwrSAGetCoreId("`**`mycore`**`");`

`[ ]`	Square brackets enclose optional items. For example:
	`clean [-pch]`
	If you must type a square bracket as part of the syntax, it is enclosed in single quotes. For example:
	`'['--use-vector']'`

`{ }`	Braces enclose a list from which you must choose one or more items. For example:
	`add {`*`signalPattern`*` | `*`portPattern`*`} `*`ID`*
	If you must type a brace as part of the syntax, it is enclosed in single quotes. For example:
	`DECLARE '{'`
	`Item1`
	`Item1`
	`'}'`

`|`	A vertical bar separates items in a list of choices. For example: `autoflush {on | off}`
>	A right angle bracket separates menu commands. For example: The *Project* > *Update System Library* menu command is available.

`...`	A horizontal ellipsis in syntax indicates that the preceding expression may have zero, one, or more occurrences. For example:
	`build-options -`*`option optionArgs`*` ...`
	A horizontal ellipsis in examples and system messages indicates material that has been omitted. For example:
	`::scsh> dtrace add top1.signal_* $t1`
	`::scsh> dtrace add top1.clk_* $t1`
	`...`
	`::scsh> dtrace flush *`

< > < >

1.3. Terminology

API	Application Programmer's Interface
DLL	Dynamic Link Library
IDE	Integrated Development Environment
TLM	Transaction-Level Modeling
VDK	Virtualizer Development Kit
VP	Virtual Prototype

1.4. Customer Support

For technical support (regarding license keys, IP downloads, Host ID, Project ID, documentation or general support), contact the Support Center with a description of your question and supplying the debug information, using one of the following methods:

- Go to https://solvnetplus.synopsys.com and sign-in with your Synopsys SolvNetPlus credentials. Select *Cases* from the menu bar, and select *Create a New Case*. Provide the requested information, including:
  - **Product L1:** Virtual Prototyping.
  - **Product L2:** Select the product type that closest matches yours.
  - **Case Type**: Select the case type from the drop-down menu.
  - **Case Severity:** Select the case severity from the drop-down menu.
  - **Subject:** Provide a brief summary of the issue or list the error message you have encountered.
  - **Description:** For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood.

  After creating the case, attach the debug files you have created, if any.
- Or, send an e-mail message to vp_support@synopsys.com. (your email will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):
  - Include the Product name, Sub Product name, and Tool Version number in your email; so that it can be routed correctly.
  - For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood.
  - Attach any debug files you created in the previous step.
- Or, telephone your local support center:
  - North America:

    Call 1-800-245-8005 from 7:00 AM to 5:30 PM Pacific time, Monday through Friday.
  - All other countries:

    https://www.synopsys.com/support/global-support-centers.html

1.5. Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

2. Chapter 1
Use Cases for Virtualizer Studio

Virtualizer Studio has the following primary use cases:

- TLM Creation
- VDK Creation
- VDK Debug
- VDK Creator Terminology

Each use case is driven by an underlying Virtualizer technology which comes pre-installed within Virtualizer Studio, and in some cases in conjunction with capabilities from other Eclipse plug-ins already pre-installed in Virtualizer Studio.

Virtualizer Studio also provides a dedicated Eclipse perspective for each of the three primary use cases, providing a suggested default layout of the different views typically desired for each use case. As with any Eclipse perspective, you can modify this arrangement of views to suit your custom needs.

< > < >

2.1. 1.1 TLM Creation

This use case is primarily based upon the TLM Creator technology in Virtualizer. For information on how to use this technology within the context of Virtualizer Studio, see the *TLM Creator User Guide*.

< > < >

2.2. 1.2 VDK Creation

This use case is primarily based upon the VDK Creator technology in Virtualizer. For information on how to use this technology within the context of Virtualizer Studio, see the *VDK Creator User Guide*.

< > < >

2.3. 1.3 VDK Debug

This use case is primarily based upon the VP Explorer technology in Virtualizer. For information on how to use this technology within the context of Virtualizer Studio, see the *VP Explorer User Guide*.

< > < >

2.4. 1.4 VDK Creator Terminology

This section describes the following VDK Creator Terminologies:

- Virtualizer Development Kit (VDK)
- VDK Project
- VDK Building Block
- Fixed VDK
- Extensible VDK
- Example VDK
- Multi-Node VDK
- VDK Node
- Multi-Node VDK Project
- VP Config
- TLM Model
- TLM Project
- Extensible TLM
- TLM Building Block

1.4.1 Virtualizer Development Kit (VDK)

Virtualizer Development Kit (VDK) is a general term for a complete collection of items required in order to make use of a Virtual Prototype, typically for the purpose of embedded software development or validation. This collection includes the Virtual Prototyping simulation itself, along with all IP models, embedded software to run on the processors in the simulation, and all necessary artifacts such as scripts, external applications and so on.

A VDK can exist in different forms, such as the output of a locally built VDK project, a pre-packaged Fixed VDK, or a collection of multiple VDKs combined into a Multi-Node VDK.

1.4.2 VDK Project

A VDK project is an Eclipse project, which forms the source code of a VDK. The primary role of VDK Creator is to allow you to edit their VDK designs, and subsequently build those VDK projects, in order to generate a runnable VDK.

You are required to have the full version of the Virtualizer tool chain, along with an appropriate Virtualizer license in order to use VDK projects.

For more information, see the "VDK Projects and Files" section of the *VDK Creator User Guide*.

1.4.3 VDK Building Block

A VDK Building Block is a reusable subsystem, that forms a part of a VDK design. This can exist in two forms, a source project, and a pre-built IP library.

VDK Creator provides you with the ability to create and edit VDK Building Block projects, and subsequently build those projects, in order to generate an IP library for this VDK Building Block. This IP library is then typically used in other VDK, or VDK Building Block designs.

1.4.4 Fixed VDK

A Fixed VDK is a pre-built VDK, that contains only the files required to run the VDK, and does not contain the source VDK project. A Fixed VDK is typically packaged into a zip file, which gets distributed by the user, developing the VDK, when handing off to the user, using the VDK for software development or validation. The developer may optionally provide an installer for the Virtualizer Runtime tool chain along with the Fixed VDK package.

A Fixed VDK gets created and packaged by building a VDK project, using the full version of the Virtualizer tool chain, along with an appropriate Virtualizer license.

A Fixed VDK package can be used by any user who has a Virtualizer Runtime tool chain, along with the required runtime license for that VDK.

For more information, see the "Running Fixed VDKs" section of the *VDK Creator User Guide*.

1.4.5 Extensible VDK

An Extensible VDK is a VDK project that can be provided to another user, and used as a starting point for their own derivative VDK project.

One example of an Extensible VDK is the VDK Starting Points that are provided as part of the Virtualizer tool chain. The typical use case is that a user creates their new VDK project as a derivative of the most appropriate VDK Starting Point, as opposed to starting with a completely empty design.

Another example of an Extensible VDK is a more complete VDK Design, which is provided as a source project for another user to modify or to extend.

1.4.6 Example VDK

An Example VDK is a source VDK package that is typically provided as an example, which you can open, browse, modify and rebuild. Example VDKs are very similar to Extensible VDK in a way that you create a new project that is a derivative of the original project. However, the Example VDKs are listed differently, and are often shipped as part of IP libraries, to provide you the IP library with a simple example of how this IP can be used in the context of a VDK, along with some simple software to run on this Example VDK.

1.4.7 Multi-Node VDK

A Multi-Node VDK is a VDK that comprises of multiple individual VDK simulations (called *VDK Nodes*), and can also include third-party simulations (called *External Nodes*). Those nodes can communicate with each other, and are subjected to a common synchronization and life-cycle control provided at the higher Multi-Node VDK level.

1.4.8 VDK Node

A VDK Node is the name given to an individual (Fixed) VDK simulation which comprises a Multi-Node VDK. Third-party applications or simulations are referred to as *External Nodes*.

1.4.9 Multi-Node VDK Project

A Multi-Node VDK project is an Eclipse project, which defines the design of a Multi-Node VDK. The complete design of the Multi-Node VDK is often an aggregation of design information distributed across multiple layered project and configuration files contained within the Multi-Node VDK project.

The Multi-Node VDK project can be edited, packaged and distributed to another user, and can be used to launch a Multi-Node VDK.

1.4.10 VP Config

A VP Config is a collection of files and settings, used to encapsulate a runtime use case for the VDK. A typical example is that a VP Config provides a software image that can be run on the simulation, along with a set of configuration settings, parameters and scripts that enable the software to run correctly on this simulation.

A VDK or Fixed VDK often consists of multiple VP Configs. You select which VP Config you wish to run when launching a VDK. VP Configs can also be packaged individually and shared between users, and can also be distributed along with an Extensible VDK, Example VDK or a Fixed VDK.

For more information, see the "Using VP Configs" section of the *VP Explorer User Guide*.

1.4.11 TLM Model

A TLM model is a building block for VDK designs, typically representing a peripheral within a simulation. A VDK developer has access to a library of TLM models, with each individual IP library containing a runtime implementation in the form of a DLL, along with information on interfaces and parameters, used by VDK Creator when instantiating and connecting this building block to a VDK design.

1.4.12 TLM Project

A TLM Project is an Eclipse project, containing the design information and source code implementation of a TLM model. This project can be edited by using TLM Creator, and subsequently built and packaged into an IP library to be distributed or made available to the VDK developer.

For more information, see the "TLM Project Editor" section ofthe *TLM Creator User Guide*.

1.4.13 Extensible TLM

An Extensible TLM is a TLM project that can be provided to another user as a source project, in order to allow them to modify and rebuild the TLM model.

One example of using Extensible TLM as an optional source package that can be shipped as part of an IP library, alongside the pre-built version of the TLM model. This allows a VDK developer who is using this TLM model in their design to rebuild the TLM model locally and run their locally rebuilt version instead.

Another example of where an Extensible TLM can be used is an Example TLM, which is a standalone package containing a source TLM project for a particular model, but not necessarily as a sibling to the pre-built IP library.

1.4.14 TLM Building Block

A TLM Building Block is a granular modeling object which can be used as a part of the design of a TLM Model. They can typically exist in several different forms, and are available to TLM developers as a part of their modeling library.

1.4.14.1 Protocol Engine

A protocol engine is a form of TLM Building Block that represents a model interface. Each type of interface has a different protocol engine, and there are sub-variants depending on the direction or role of the interface. The protocol engine not only provides the means to add the interface to the model, but assists with the generation of the source code to use the necessary sockets or implementation objects, perform binding, initialization, and assists you with implementing callback functions to implement behavior, as well as providing counterpart objects in the mirror model to facilitate the test code development.

For more information, see the "Protocols" section of the *TLM Creator User Guide*.

1.4.14.2 Modeling Object

A modeling object is a form of TLM Building Block that represents a module of functionality. This can be a very primitive object such as a variable or an event, up to slightly more complex objects such as FIFO, all the way up to complex objects such as a DMA engine. Modeling objects are analogous to classes in C++, in that they encapsulate functionality to enable reuse. They also provide you with well defined entry points to configure the object, invoke behavioral APIs, and implement callbacks.

For more information, see the "TLM Building Blocks" section of the *TLM Creator User Guide*.

1.4.14.3 Common Settings Object

A common settings object is a form of TLM Building Block that simply provides project settings into a TLM project. Individual TLM projects can select any number of common settings objects to be applied to this project. This may be used to provide a consistent set of build options to be applied commonly across a set of TLM models, or it may be used to provide a specific set of functionality, such as customized documentation or test code generation.

For more information, see the "Common Settings Building Blocks" section of the *TLM Creator User Guide*.

---

3. Chapter 2
Getting Started with Virtualizer Studio

This chapter describes the key steps required to start with Virtualizer Studio.

- Configuring the Virtualizer Environment
- Launching the Virtualizer Studio IDE
- Selecting the Eclipse Workspace
- Virtualizer Studio Welcome Screen
- Example Browser
- GUI Icons
- Launching the Virtualizer Studio Shell Application
- Command-Line Arguments

3.1. 2.1 Configuring the Virtualizer Environment

Prior to launching Virtualizer Studio, setup your Virtualizer environment in the same way as for all Virtualizer tools. This process includes calling the main Virtualizer setup script, and providing information on the product and license level being used. For further information on how to run the setup scripts, see the *Synopsys Virtual Prototyping Product Installation Guide* (`PAVP_Install.pdf`).

The following table shows the relevant configuration of the environment, along with details on which licenses will be used, and which use cases will be enabled:

Table 2-1 Configuring the Virtualizer Environment

Setup Script Arguments	Remarks	Licenses		Use Cases		
		Virtualizer	VDK User	TLM Creation	VDK Creation	VDK Debug
`-vzb` or `-vze`	Uses a Virtualizer license. Enables all Virtualizer Studio use cases.	X		X	X	X
`-tvb` or `-tve`	Uses a Tools for VDK license. Enables VDK Debug use cases.		X			X

3.2. 2.2 Launching the Virtualizer Studio IDE

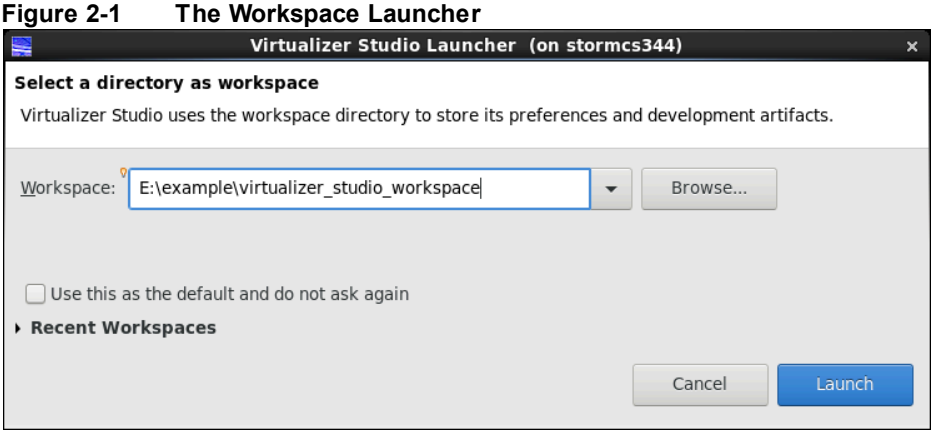
Once the Virtualizer environment has been properly configured, launch Virtualizer Studio by using any one of the following commands depending on your platform:

- On Windows: `vs.exe`
- On Linux: `vs`

3.3. 2.3 Selecting the Eclipse Workspace

Eclipse IDEs typically use the concept of an Eclipse workspace, as being the area where your projects are stored on disc. Upon launching an Eclipse IDE, you are prompted to select the location of your workspace, as shown below.

Figure 2-1 The Workspace Launcher

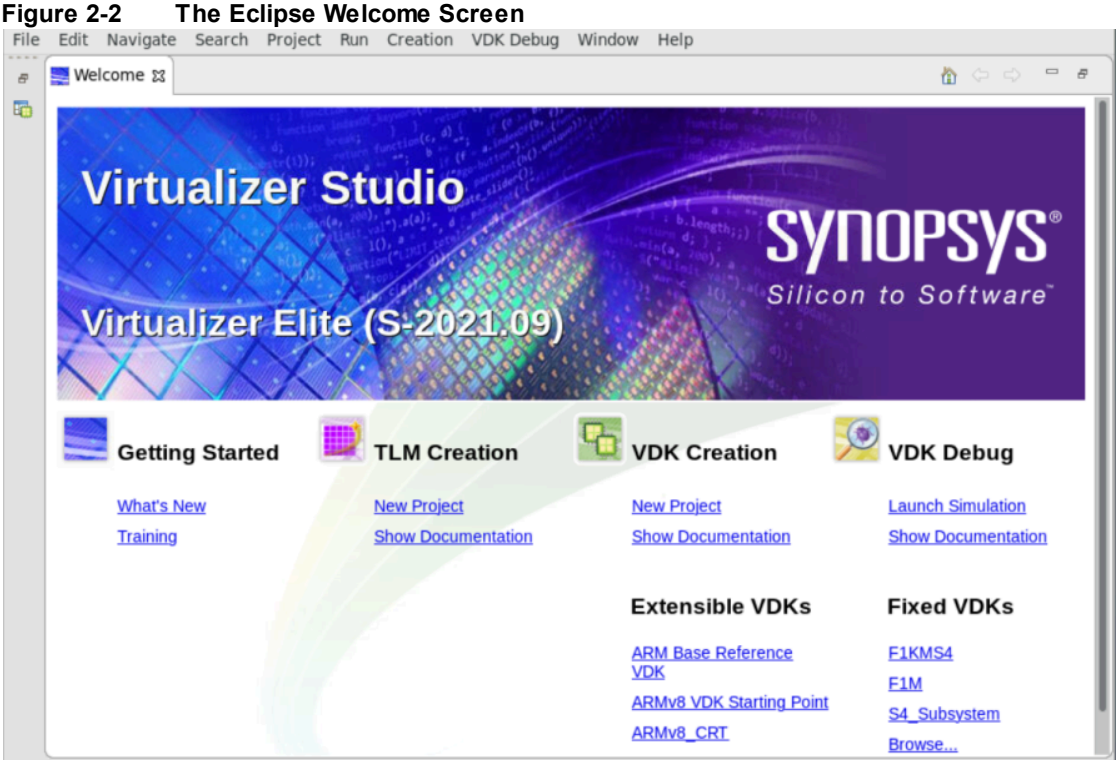


You can either set it to point to a directory which does not yet exist, in which case a clean workspace will be created at that location, or can set to point to an existing workspace, in which case it will reopen the previous working environment.

3.4. 2.4 Virtualizer Studio Welcome Screen

The first time Virtualizer Studio is opened with a clean workspace, the initial screen you will see is the Eclipse Welcome screen, as shown below. Note that the release version displayed on this screen depends on the installer version.

Figure 2-2 The Eclipse Welcome Screen



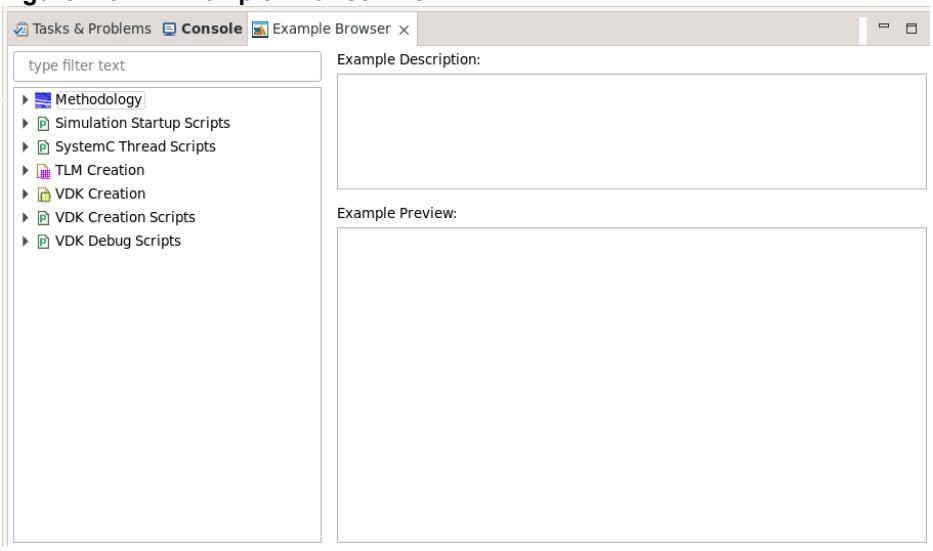
Virtualizer Studio's Welcome screen contains hyperlinks to the most common starting points for each of the primary use cases for Virtualizer Studio. This will typically be the creation of a new project, viewing the documentation, or launching a VDK. In addition, links will also appear to create new derivative VDK projects based on a Reference VDK, if any such Reference VDKs are available in your environment.

The Welcome screen will automatically close after creating your first project. From the menu bar, select *Help* > *Welcome* to reopen the screen again.

3.5. 2.5 Example Browser

Virtualizer Studio has an integrated browser to find example content that is presented in the *Example Browser* view which is shown next to the *Console* view by default, as shown below.

Figure 2-3 Example Browser View

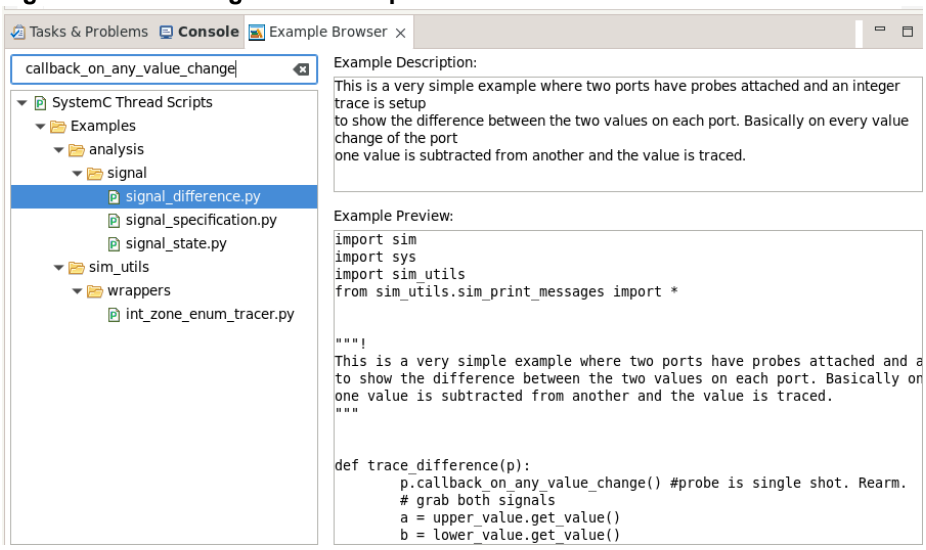


It contains several examples for different use flows around Virtualizer Studio that are presented in high-level categories like TLM Creation, VDK Creation and several scripting related categories.

When selecting an example from the list, a description and, if available, a preview of the example is presented in the Example Browser. Depending on the type of the example, different right-click context menu options exist to use or open the example.

The filter text box can be used to search for specific examples not only by their name, but also by inspecting the content of example scripts. The following figure shows how you can use the Example Browser.

Figure 2-4 Using in the Example Browser



3.6. 2.6 GUI Icons

Virtualizer Studio has a set of icons on the main IDE toolbar, which are used across throughout the different use flows in the tool. The following table presents the description of the icons.

Table 2-2 Project Icons

Icon	Description
	Is a VDK Creator project (opened).
	Is a TLM Creator project (opened).

Table 2-3 File Type Icons

Icon	Description
	Is a VDK Creator file with an extension `.vdksys`.
	Is a fixed VDK file with an extension `.vdkfxd`.
	Is an external simulation Interface with an extension `.vdkxif`.
	Is a TLM Creator file with an extension `.tlmc`.
	Is a VP Config file with an extension `.vpcfg`.

Table 2-4 Application Tool Bar Icons

Icon	Description
	Creates a new project.
	Saves the modified resource.
	Saves all modified resources.
	Debug is the standard Eclipse Debug button, and will launch a simulation for a TLM unit test or a VDK's VP Config with both the VDK debugger and the source code debugger attached.
	Run is the standard Eclipse Run button, and will launch a simulation for a TLM unit test either with or without the VDK debugger connected, or will launch a VDK's VP Config (as selected in the *Active VP Config Selection*) with the VDK debugger attached. **NOTE**: In cases where there is no active context available, *Run* reverts to its history and will run the last configuration which was run.
/	*Pause and Continue Simulation* provides simulation control over the simulation currently running and under the control of the VDK debugger. **NOTE**: The icon for this button will switch depending on whether the VDK debugger is suspended or free running.
	Configure Analysis will open the Analysis tab in the VP Config editor, in order to allow you to view and configure the analysis which is currently enabled on this configuration.
	Source Code Debug will launch and connect the source code debugger to the running simulation. On Linux, this will open GDB inside Virtualizer Studio. On Windows, this will open Microsoft Visual Studio if it installed and available in your environment.

	Stop Simulation will terminate the running simulation.
	Restart Simulation will restart the previous simulation which was run in this Virtualizer Studio session. **NOTE**: This is independent of the selection of the active VP Config, and may in some cases result in launching a different simulation or configuration than the *Run* button.
	Opens a new perspective.
	Opens the VDK Creation perspective.
	Opens the TLM Creation perspective.
	Opens the VDK Debug perspective.
	Shows the Project Explorer. *Home* is a shortcut to return back to the VDK project, based on the currently active VP Config. This will switch back to the correct perspective and also open the correct project file in the editor.
	Build will build the TLM or VDK project which is currently in focus. The active project is determined by the view which has the focus. For example, if the editor is in focus, then the file currently open in the editor will determine the active project, and if the Project Explorer view has the focus, then the selection within the Project Explorer will determine the active project. The *Clean* operation can also be triggered from this button, as one of the options in the button's drop-down menu.
	Package will allow you to generate a package, or all packages, for the active project. VDK projects maintain a list of VDK packages which can be generated, as well as a patch for a Fixed VDK.
	Edit VP Config allows you to open the active VP Config in the editor, and also provides you with the ability to create a new VP Config via the button's drop-down menu.
Mini-AArch64-Linux	The *Active VP Config* combo-box allows you to select which VP Config should be launched on clicking *Run*. This also has an impact on the runtime values displayed in a VDK project. The drop-down menu shows a list of all VP Configs which are part of the active VDK project, allowing you to select the one to become the active VP Config. While a simulation is running, the combo-box will display the VP Config of the simulation which is currently running.

Note	Some buttons will be disabled at times when their use is not relevant, or when no appropriate context is available.

Icons appearing on the user interface of TLM Creation, VDK Creation, and VDK Debug perspectives are explained in the respective user guides. For details, see the section "GUI Icons" in the *TLM Creator User Guide*, *VDK Creator User Guide*, and *VP Explorer User Guide*.

< > < >

3.7. 2.7 Launching the Virtualizer Studio Shell Application

Once the Virtualizer environment has been properly configured, launch Virtualizer Studio Shell by using any one of the following commands depending on your platform:

- On Windows: `vssh.exe`
- On Linux: `vssh`

When launched without specifying an Eclipse workspace, a default workspace will be selected in your home directory.

When launched without specifying a script to be run, Virtualizer Studio Shell will offer you the ability to interact with the Python interpreter.

A typical usage of Virtualizer Studio Shell would be in a regression environment, where a Python script will be passed in to perform a sequence of actions. An example of a command to run such a scenario would be:

```
vssh -d ./my_workspace -s ./my_script.py
```

For a list of command-line arguments available in Virtualizer Studio Shell, see the subsequent section "Command-Line Arguments" on page 23.

3.8. 2.8 Command-Line Arguments

The following command-line arguments are supported by both the `vs` and `vssh` applications, unless stated otherwise:

Table 2-5 Command-Line Arguments

Flag	Description	Comments
`-s` `--script`	Executes a Python script.	Followed by the file path.
`--pyargs`	Lists arguments that are passed to the Python script. Can use `--pyargs_end` to indicate the end of Python arguments.	
`-d` `--data`	Specifies path to Eclipse workspace.	Followed by the directory path.
`-c` `--configuration`	Specifies path to Virtualizer Studio cache directory.	Followed by the directory path.
`-r` `--run`	Runs a VP Config.	Followed by the file path.
`--clean`	Forces Virtualizer Studio to reinitialize its plug-in caches. If you add third-party plugins or receive updates to your installation, you may want to use the `--clean` option to force the Eclipse runtime to purge its caches used to store plug-in dependency information.	
`--cleanall`	It is similar to `--clean` flag. Additionally, wipes all user configurations, and potentially deletes installed third-party plug-ins.	
`--vmargs`	All options after this flag are passed to the Java VM.	These options must be the last ones on the command line.
`-v` `--version`	Prints the details on version.	
`-h` `--help`	Shows information on the usage.	
`--vpx_tcl`	Executes a TCL script using the VP Explorer TCL interpreter.	Followed by the file path.

`--tlmc_tcl`	Executes a TCL script using the TLM Creator TCL interpreter.	Followed by the file path.
`--logfile`	Writes `stdout` and `stderr` from Virtualizer Studio to a log file.	Followed by the file path.
`--pyout`	Writes output from Python interpreter to a log file.	Followed by the file path.
`--tclout`	Specifies log file for Tcl interpreter output.	Followed by the file path.
`--initdir`	Starts Virtualizer Studio in the selected directory.	Followed by the directory path.
`--local`	Enables connection to simulations on the local machine only. **Note:** This option can be used to work around `vpsession` connection problems.	
`--perspective`	Starts Virtualizer Studio in one of the supported perspectives: • `tlmc`: TLM Creation • `vdkc`: VDK Creation • `vdkd`: VDK Debug • `pd`: Platform Debug • `analysis`: Platform Analysis • `debug`: CDT Debug • `cpp`: CDT C++	Followed by the perspective identifier.
`--product`	Specifies the product identifier: • `Virtualizer-Base` • `Virtualizer-Elite` • `Tools-VDK-Base` • `Tools-VDK-Elite` • `Tools-VDK-Regr-Base` • `Default`: Taken from the `SNPS_VP_PRODUCT` variable, or if not set: `Tools-VDK-Base`.	
`Flag: --bb_test_*`	Flags related to running the Virtualizer Studio Building Block Compliance Tests. For further details about the available flags and their usage, see "Command-Line Arguments" on page 61.	
`--roaming`	Enables connection to simulations also on the network. Default is to only allow the local host.	
`--simout`	Specifies log file for combined simulation output.	Followed by the file path.
`--vdk_test`	Executes a Python *<TestScript>* based on an optional *<VDK>* and a *<VPConfig>*.	This command-line argument is applicable only for `vssh`. Add arguments as a quoted list "*<VDK>*,*<VPConfig>*, *<TestScript>*". The *VDK* is optional and can be replaced by null.

		For further example, see `SLS/any/examples/VDKTestExample/README` in the installation.

`--force-debug (on|off)`
 	
`--force-optimize (on|off)`
 	
`--force-assertions (on|off)`
	Allows to override the debug/optimize/assertion settings of projects.	

4. Chapter 3 Python Console

Virtualizer Studio provides a Python console, which allows you to interact with the different underlying technologies via a command-line interface. For more details on the Console view, see "Console View" on page 29. The currently supported version is Python `3.10.4`.

Note	Previous versions of Virtualizer Studio were using a Python 2 interpreter. For detailed information about migration from Python 2 to Python 3, see "Migrating Scripts to Become Compatible with Python 3" on page 30.

DEPRECATED: Python 2 support is deprecated in Product Version T-2022.06 and will be removed in the next major release.

This chapter describes:

- Running Scripts
- Context-Sensitive Help
- Python Objects
- APIs
- Console View
- Migrating Scripts to Become Compatible with Python 3

4.1. 3.1 Running Scripts

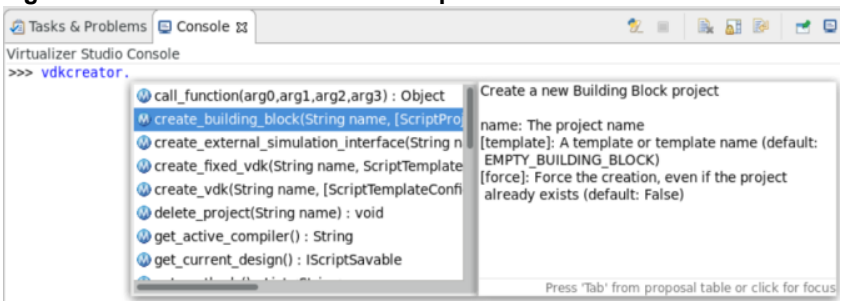
Python scripts can be executed by right-clicking on them in the Project Explorer view and selecting *Run in Console*, or by dragging and dropping them from the Project Explorer view into the Virtualizer Studio Console.

4.2. 3.2 Context-Sensitive Help

A list of available APIs will appear when attempting to call a class member function of a known Python object. This allows you to see all available calls on that object along with a description, and information on the return type and parameters. It also provides the capability of auto-completing your entry.

Hint	Context-Sensitive Help will appear as you type, but can also be manually activated by pressing Ctrl+Space.

Figure 3-1 Context-Sensitive Help



Note	Context-Sensitive Help is not available on an object which is returned by another call within the same line. The recommendation in this case is to split the call into multiple lines, storing the return of the first call into an object to be used in the next line.

< > < >

4.3. 3.3 Python Objects

When interactively typing Python code into the console, many APIs will return Python objects, which can be stored into named variables to be reused in subsequent lines.

Example:

In the following example, the first line will assign an object referring to the current VDK design into a variable called `vdk`. The second line will call an API belonging to this type of object, in this case it will call the `build()` API.

```
vdk = vdkcreator.get_current_design()
vdk.build()
```

Hint	You may often execute a command in Python without storing the returned object. There is a way to retrieve the last object returned by referring to it using a single underscore, and retrospectively assign it to a named variable, without the need to re-execute the previous command. The following two commands are equivalent to the first command in the example above: `vdkcreator.get_current_design()` `vdk = _`

< > < >

4.4. 3.4 APIs

For information on the available APIs, see the following manuals:

- *Virtualizer Studio Python Interface Reference Manual* describes the Python interface of Virtualizer Studio.
- *TLM Creator Tcl Interface Reference Manual* describes the Tcl interface of TLM Creator.
- *VP Explorer Tcl Interface Reference Manual* describes the Tcl interface of VP Explorer.
- *VP Explorer Python Interface Reference Manual* describes the Python interface of VP Explorer.

< > < >

4.5. 3.5 Console View

The *Console* view in Virtualizer Studio is a general eclipse view, providing access to many different consoles available in the IDE.

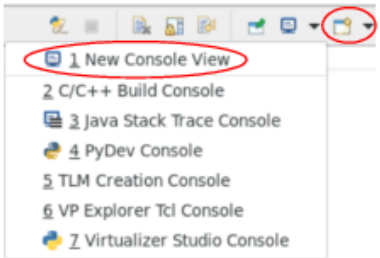
It has buttons on its toolbar for *Open Console* and *Display Selected Console* actions. Both these buttons have a drop-down menu which lists the available consoles, allowing you to switch between different consoles. The toolbar also contains a *Pin Console*

option, which is useful if you wish to keep the Console view showing one specific console.

3.5.1 Creating Multiple Console Tabs

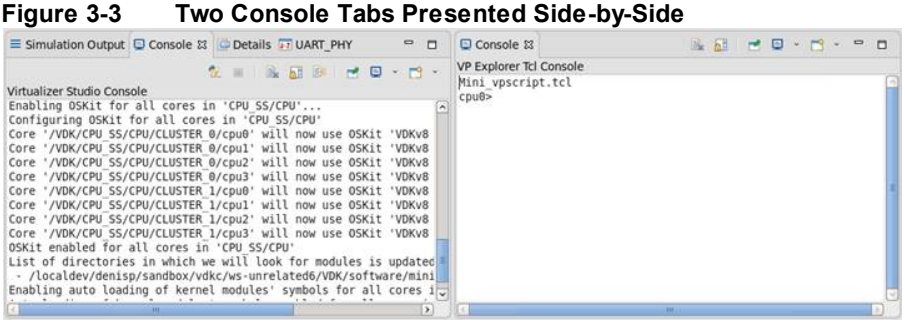
The *New Console View* option in the drop-down menu of the *Open Console* button will open a second instance of the *Console* view in another tab.

Figure 3-2 Selecting New Console View Option



This tab can then be configured independent of the original instance of the Console, and can be dragged around the IDE, or moved outside the IDE, like any other *Eclipse* tab. In the example below, the two console tabs have been configured to show a different console, and have been located side by side for convenience, as shown below.

Figure 3-3 Two Console Tabs Presented Side-by-Side



3.5.2 Available Consoles

The following table describes the consoles that are available in Virtualizer Studio.

Table 3-1 Available Consoles

Consoles	Descriptions
Virtualizer Studio Console	Python interpreter preconfigured with the ability to execute all relevant Virtualizer Studio APIs. This is the recommended scripting interface for interacting with VDK projects, VP Configs, running simulations and general Eclipse projects.
TLM Creation Console	Tcl interpreter preconfigured with the ability to execute TLM Creator Tcl APIs. This is the recommended scripting interface for interacting with TLM projects.
VP Explorer Tcl Console	Tcl interpreter preconfigured with the ability to execute VP Explorer Tcl APIs. This has the ability to interact with VP Configs and running simulations. However, the Python interpreter in the Virtualizer Studio console is the recommended interface for use.
C/C++ or CDT Build Console	Standard console provided by CDT (C/C++ Development Tooling). There will typically be a CDT Global Build Console, as well as a project specific CDT Build Console. This provides the output from the build of a CDT project, including embedded software projects, and TLM projects.
Virtualizer SystemC GDB	Provides an interface to the GDB SystemC debugger, connected to the running simulation. This is typically used to debug TLM models and SystemC code. Typically, it consists of two consoles, one containing output traces from GDB, and one allowing for GDB commands to be sent at times when GDB is suspended.
GDB Hardware Debugging	Provides an interface to the GDB hardware debugger, connected to a CPU or a cluster of CPUs, running inside the running simulation. This is typically used to debug embedded software running on

	the VDKs. Typically, it consists of two consoles, one containing output traces from GDB, and one allowing for GDB commands to be sent at times when GDB is suspended.
PyDev Console	Is the console provided by the PyDev plugin. You will need to configure the PyDev interactive console settings from the Preferences dialog prior to using this console.

Other consoles may become available after installing third-party plugins into the IDE.

< > < >

4.6. 3.6 Migrating Scripts to Become Compatible with Python 3

Python 2 has reached its end of life and is no longer actively maintained. Virtualizer Studio provides a Python 3 based interpreter.

Since Python 3 does not maintain full backwards compatibility with Python 2, a Python 2 compatible script can break in a Python 3 interpreter and vice versa. A list of Python 2 and Python 3 differences can be found here *http://python3porting.com/differences.html*.

There are multiple packages such as *futurize*, *modernize* or *six* which help to migrate existing scripts and provide a compatibility layer to close the gap between Python 2 and Python 3. The future package is now shipped together with Virtualizer Studio.

3.6.1 Using Futurize to Upgrade Existing Scripts

3.6.1.1 Concept of Futurize

The *futurize* is a small wrapper around the *2to3* feature of Python and helps with automatic code conversions. There are multiple fixers which can be applied, they are grouped into two stages.

- Stage 1

  This is for safe changes, which simply modernize existing code without breaking compatibility with Python 2.6 or introducing a dependency to the future package. The goal is to handle most of all required changes which do not need detailed review.

- Stage 2

  This applies further fixers, which transform the code into more Python 3 style code. These transformations will also introduce a dependency to the future package, which provides additions from Python 3 and shadows certain Python 2 built-ins to provide their corresponding Python 3 semantics. These imports will not have any effect in a Python 3 interpreter.

For being able to use the *futurize* script, Python needs to be setup correctly:

```
#setup python
cd ${SNPS_VP_HOME}/python/<compiler>
source python_setup.sh
cd -
```

3.6.1.2 Applying the More Straightforward Upgrades

With *futurize* on the path, it is now possible to automatically transform existing python files. For applying all Stage 1 fixers, the following command will perform all transformations in-place:

```
futurize --stage1 -w -n <path_to_python_dir_or_file>
```

Example:

```
#before
try:
   call_something()
   print "ok"
except Exception, e:
   print e
------

#after
from __future__ import print_function
try:
   call_something()
   print("ok")
except Exception as e:
   print(e)
```

The `-w` flag will write the changes back into the source file, `-n` flag specifies that no backup files should be written.

It is recommended to perform the automatic transformations in a revision-controlled environment, to be able to easily review or revert the applied changes. Otherwise, it is recommended to create a backup copy of the file first.

In certain cases, it might make sense to even split up stage 1 into multiple commits to have a better overview over the applied changes. For example, to only perform the `print` function refactoring one can execute this single fixer with:

```
futurize --stage1 -w -n <path_to_python_dir_or_file> -f
libfuturize.fixes.fix_print_with_import
```

A list of all fixers can be printed with:

```
futurize -l
```

In case certain transformations should explicitly not be applied, they can be excluded with the `-x` flag. For further options, see the command-line help of the script.

With stage 1 fixers, most of the required changes should have been handled already without introducing a dependency on the future package.

3.6.1.3 Applying the More Complex Upgrades

Stage 2 of the conversion process will re-factor the code to more Python 3 style concepts. This introduces imports from the future package to make these concepts available in a Python 2 interpreter. Virtualizer Studio will also ship the future package alongside the Python 3 interpreter in a future release. For example, the standard library has been reorganized (see PEP 3108) and the `libfuturize.fixes.fix_future_standard_library` fixer installs aliases to make the Python 3 imports work in Python 2.

Note	Automatic code conversion is just a tool to assist in the migration process, it will not handle all possible cases and in rare cases can even lead to incorrect code. For example, one fixer (`lib2to3.fixes.fix_itertools_imports`) correctly transforms to Python 3 code, but is missing the future import to remain Python 2 compatible. This can happen, because *futurize* is built on top of *2to3,* which only aims to migrate a codebase to Python 3. A manual addition of the standard library aliases will make it Python 2 compatible again. It is recommended to manually review the changes done by the individual fixers.

Example:

```
#before
import ConfigParser
n = raw_input('Enter a number:\n')
for i in xrange(1,n):
   print('number is: ' + str(i))
print(range(1,5))
m = {"a":1, "b":2}
for k,v in m.iteritems():
   print("{}->{}".format(k,v))
-----

#after
from __future__ import print_function
from future import standard_library
standard_library.install_aliases()
from builtins import input
from builtins import str
from builtins import range
import configparser
n = input('Enter a number:\n')
for i in range(1,n):
   print('number is: ' + str(i))
print(list(range(1,5)))
m = {"a":1, "b":2}
for k,v in m.items():
   print("{}->{}".format(k,v))
```

3.6.2 Additional References

The following are the reference documentation that can be found online for Python 3 migration:

- *https://python-future.org/futurize.html*
- *http://python-future.org/compatible_idioms.pdf*
- *http://python-future.org/conversion_limitations.html*
- *https://docs.python.org/3/howto/pyporting.html*

- http://blog.pyspoken.com/wp-content/uploads/2018/07/python2_to_3_migration_guide.pdf

5. Chapter 4
IP and VDK Packaging Flows

Virtualizer Studio provides various different flows for building and distributing IP libraries and VDKs.

This chapter describes:

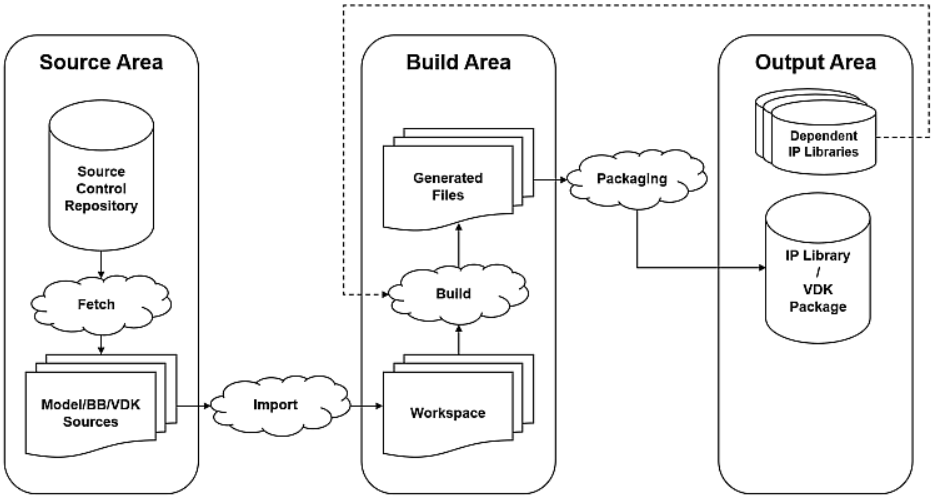
- Overview
- TLM Building Blocks
- TLM Models
- VDK Building Blocks
- Virtualizer Development Kits
- Fixed VDKs

5.1. 4.1 Overview

Several different flows of building and distributing IP libraries and VDKs are supported in Virtualizer Studio. Those flows are in general referred to as *packaging,* which describes the process of collecting and exporting all files that will be required to be shared with a receiving user. This process of providing the output of one user's work as an input to the next user should be applied independent of whether users are working in the same or different companies or teams within a company.

In the following figure, *Packaging* refers to the action of taking the build output of a project and exporting it into the output area.

Figure 4-1 Packaging Block Diagram

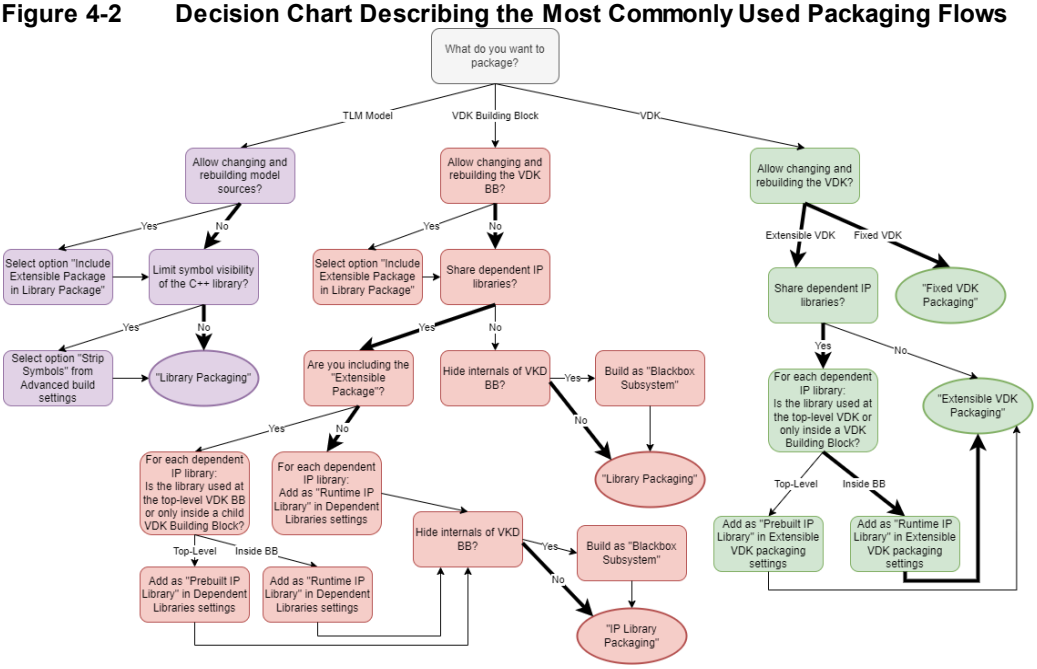


Virtualizer Studio supports packaging flows for the following project types, each of which will be covered in more detail in a dedicated section below:

- TLM Building Blocks
  - Library Packaging: Creation of a pre-built TLM Building Block library.
  - Test Packaging: Creation of a test package that can be used to execute unit tests.
- TLM Models
  - Library Packaging: Creation of a pre-built TLM Model library.
  - Extensible Packaging: Creation of an Extensible TLM Model.
  - Test Packaging: Creation of a test package that can be used to execute unit tests.

- VDK Building Blocks
  - Library Packaging: Creation of a pre-built VDK Building Block library.
  - Extensible Packaging: Creation of an Extensible VDK Building Block.
  - IP Library Packaging: Creation of a package containing the VDK Building Block and all its dependent IP libraries (that is, other TLM Models and/or VDK Building Blocks).
- Virtualizer Development Kits
  - Fixed VDK Packaging: Creation of a pre-built VDK simulation.
  - Extensible VDK Packaging: Creation of an Extensible VDK.
- Fixed VDKs
  - Fixed VDK Re-Packaging: Redistribution of a pre-built VDK simulation.

Figure 4-2 Decision Chart Describing the Most Commonly Used Packaging Flows



4.1.1 Library Management

Depending on the type of the IP library package, different mechanisms exist to make an IP library available for use in downstream projects:

- TLM Building Blocks
  - Library Package: Contains a `*.tlmclib` file which needs to be added to the library manager of downstream TLM Model projects.
- TLM Models
  - Library Package: Contains a `*.pctlib` file which needs to be added to the library manager of downstream VDK or VDK Building Block projects.
  - Extensible Package: Contains a `*.tlmclib` file.
    - The `tlmclib` file needs to be added to the library manager of downstream VDK or VDK Building Block projects to be able to open the Extensible Model from inside the VDK or VDK Building Block project.
    - The `tlmclib` file needs to be found via the *Additional search path* entry or the environment variable `SNPS_VS_TLM_SEARCH_PATHS` to become available as a template in the *New TLM Project* dialog.
- VDK Building Blocks
  - Library Package: Contains a `*.pctlib` file which needs to be added to the library manager of downstream VDK or VDK Building Block projects.
  - Extensible Package: Contains a `*.vdklib` file.
    - `vdklib` file needs to be added to the library manager of downstream VDK or VDK Building Block projects to be able to open the Extensible Building Block from inside the VDK or VDK Building Block project.
    - `vdklib` file needs to be found via the *Additional search path* entry or the environment variable `SNPS_VS_VDK_SEARCH_PATHS` to become available as a template in the *New VDK Project* dialog.
  - IP Library Package: Contains the `*.ip.zip` file which needs to be extracted by the receiving user. The content is a directory containing the VDK Building Block and all its dependent libraries in individual sub-directories. The resulting

libraries need to be added to the library manager of a design that should use any of the contained IP libraries.

- Virtualizer Development Kits
  - Fixed and Extensible VDK Package: A `*.vdk.zip` file will be created that needs to be extracted by the receiving user. The path to the extracted files needs to be added to the *Additional search path* entry or the environment variable `SNPS_VS_VDK_SEARCH_PATHS` to become available as a template in the *New VDK Project* dialog.

4.1.2 IP Library Types

When sharing IP libraries with downstream users, different types of libraries can be shared:

- Runtime Library

  This library consists of the minimal set of files needed to run a (Fixed) VDK that contains an instance of this IP library. The minimal set of files contained will typically be a `*.pctlib` file and the actual C++ library.

- Prebuilt Library

  This library contains the set of files needed to add an instance of this library into a VDK or VDK Building Block project. This is typically a superset of the Runtime Library and will contain a `*.xml` file in addition.

- Extensible Library

  This library contains the sources required to modify and rebuild the library. This is typically provided as an addition to the Prebuilt Library in cases where downstream users will potentially still have to modify the library.

The type of library to be shared can be selected when adding dependent/included libraries to VDK Building Block IP Library Packages (see "IP Library Packaging" on page 45) or Extensible VDK packages (see "Extensible VDK Packaging" on page 46).

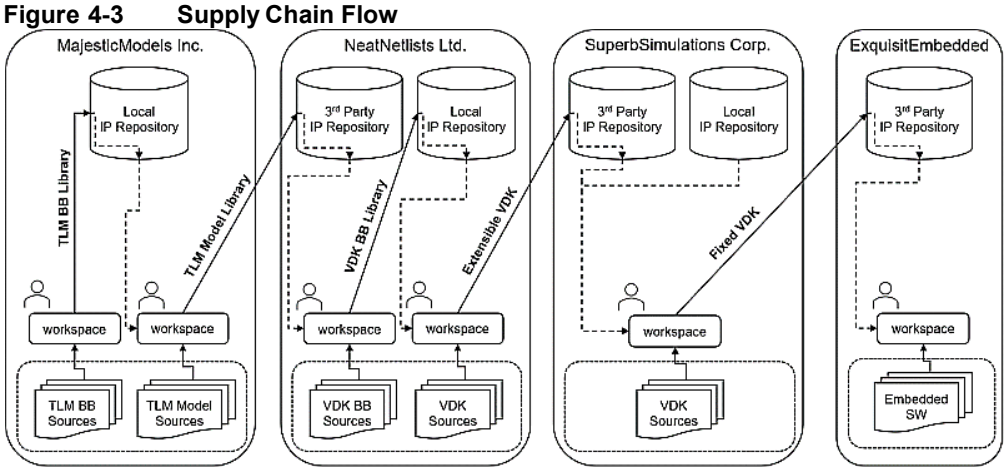
For details on use flows related to the different library types, see "Enhanced Security for Sensitive IP and VDKs" on page 65.

4.1.3 Supply Chain Example

All supported packaging flows can happen between users inside the same company or in the form of deliverables between companies. To illustrate this, an example supply chain setup between four companies is depicted below:

- MajesticModels Inc.
  - Creates TLM Building Blocks for internal use.
  - Delivers TLM Models to NeatNetlists Ltd.
- NeatNetlists Ltd.
  - Creates VDK Building Blocks for internal use.
  - Delivers an Extensible VDK to SuperbSimulations Corp.
- SuperbSimulations Corp.
  - Delivers a Fixed VDK to ExquisitEmbedded.
- ExquisitEmbedded
  - Develops Embedded Software using the Fixed VDK.

Figure 4-3 Supply Chain Flow



4.1.4 Build System Integration

Virtualizer Studio also provides support for integrating the supported packaging flows with an external build system. For more details, see "Virtualizer Studio Build System Integration" on page 49.

5.2. 4.2 TLM Building Blocks

TLM Building Blocks can be used to break down the complexity of TLM models into smaller units. They also allow functionality to be re-used across multiple TLM models. Thus, they can be seen as a regular element of an IP repository that is used during development of subsequent TLM Building Blocks or TLM models.

4.2.1 Library Packaging

Library Packaging describes the process of making a pre-built TLM Building Block available to downstream users. Following are the options for configuring a TLM Building Block library package.

- Package Content

  All required tool generated files will automatically be included in the library package. For additional contributions to the package content, the following settings are available.

Settings > *General* > *Library Packaging*	This section can be used to add custom files into the library package. The source path and the (relative) target path inside the package can be specified.

- IP Library Redistribution

  Extensible VDKs can redistribute IP libraries that are used in the VDK design in form of an IP library package that contains all required IP libraries needed for being able to re-build the Extensible VDK. A TLM Building Block package can indicate whether it allows to be redistributed, and whether it has further dependents that should also be redistributed along with it.

  Related settings that are found in the *Settings* > *Assembly* > *IP Library Redistribution* section of the *TLM Building Block* editor are listed below:

Allow redistribution of this library	Flag to control whether the TLM Building Block can be redistributed or not.
Dependent Libraries	List of dependent libraries that should also be redistributed if this TLM Building Block is included in an IP library package. Each entry in the list consists of: • *IP Library*: The name of the IP library. • *Type*: If the dependent IP library specifies multiple types, it is possible to select which one this TLM Building Block should depend on. By default, this field is empty, which means that the entire dependent library (all types) should be included when redistributing. • *Target Type*: Currently not applicable to TLM Building Blocks. It should be left empty.

4.2.2 Test Packaging

Test Packaging describes the process of exporting a TLM Building Block test package, that will enable building and executing the unit tests without requiring access to the original source project location. This is particularly useful when integrating with a build and regression system that requires the tests to be executed in a separate phase that is independent of the actual build phase.

Following are the options for configuring a TLM Building Block test package.

- Package Content

  All required tool generated files will automatically be included in the test package. For additional contributions to the test package content, the following settings are available.

Settings > *General* > *Test Packaging*	This section can be used to add custom files into the test package. The source path and the (relative) target path inside the package can be specified.

5.3. 4.3 TLM Models

The creation of Library Packages, Extensible Packages, and Test Packages is supported for TLM models. Details are explained in the subsequent sections.

4.3.1 Library Packaging

Library Packaging describes the process of making a pre-built TLM model available to downstream users. This is the regular flow for building and sharing model libraries with users that will instantiate and configure the model in a VDK or subsystem context.

An option exists to also include an extensible version (that is, a source version, see "Extensible Packaging" on page 42) of the model inside the Library Package. This is useful for flows in which the receiving user might need to re-build the model locally, for example, to build a debug version or try out changes or bug fixes in the model without having to request an updated model from the original provider.

Following are the options for configuring a TLM model package.

- Package Content

  All required tool generated files will automatically be included in the library package. For additional contributions to the package content, the following settings are available.

Settings > General > Library Packaging	This section can be used to add custom files into the library package. The source path and the (relative) target path inside the package can be specified.
Settings > General > IP Library > Include Extensible Package in Library Package	This flag controls whether the library package should also include an extensible version of the model. Enabling this, allows downstream users to open, edit and re-build a source copy of the model inside their local workspace.

  To remove all symbol table and relocation information from the generated C++ library, for example, for Secure IP Models (see "Enhanced Security for Sensitive IP and VDKs" on page 65), the option *Strip Symbols* can be enabled under *Settings > Build > Advanced*. This option is not applicable on Windows.

- IP Library Meta-Data

  The following metadata of the created IP library can be configured.

Model identifier in form of V/L/N/V (Vendor/Library/Name/Version)	• *Vendor*: Included under *Settings > General > Model Information*. • *Library*: Included under *Settings > General > IP Library*. • *Name*: Model name is used, can be changed under *Settings > General > Model Information*. • *Version*: Included under *Settings > General > Model Information*.
Path variable name	The automatically generated path variable name can be overridden under *Settings > General > IP Library*.

- IP Library Redistribution

  Extensible VDKs can redistribute IP libraries that are used in the VDK design in form of an IP library package that contains all required IP libraries needed for being able to re-build the Extensible VDK.

  A TLM Model package can indicate whether it allows to be redistributed, and whether it has further dependents that should also be redistributed along with it.

  Following related settings are included in the *Settings > Assembly > IP Library Redistribution* section of the *TLM Model* editor:

Allow redistribution of this library	Flag to control whether the TLM model can be redistributed or not.
Dependent Libraries	List of dependent libraries that should also be redistributed if this TLM model is included in an IP library package. Each entry in the list consists of: • *IP Library*: The name of the IP library. • *Type*: If the dependent IP library specifies multiple types, it is possible to select which one this TLM model should depend on. By default, this field is empty, which means that the entire dependent library (all types) should be included when redistributing. • *Target Type*: In case a library package also includes an extensible package, there can be cases where the pre-built model and the extensible model need to have different dependencies (for example, additional TLM Building Blocks might be required for re-building the extensible model). The target type can be used to indicate that a dependency only applies to a sub-type of the created library package. Leaving it empty will result in the dependency to apply for the entire library package.

Note Adding dependent libraries to those settings, will not result in them to be copied into the created IP library but instead, the dependencies will be expressed as metadata in the created IP library. This

will allow, for example, Extensible VDK packaging to collect all dependencies recursively to create an IP library package that accompanies the Extensible VDK package.

4.3.2 Extensible Packaging

Extensible Packaging describes the process of making an Extensible TLM Model package available to downstream users. An Extensible TLM Model package contains a full source version of the model, allowing users to create, edit, and re-build local copies of it.

Extensible TLM Models will be available from the New TLM Project dialog under the *Extensible TLM* project type. Selecting an *Extensible TLM Model* in this dialog will result in a local project with a copy of all the source files required to edit and re-build the model. The modified model from the local workspace project will effectively become a new model that can again be packaged and redistributed like any regular TLM model.

Following are options for configuring an Extensible TLM Model package.

- Package Content

  All required tool generated files will automatically be included in the extensible package. For additional contributions to the package content, the following settings are available.

Settings - General - Extensible Packaging	This section can be used to add custom files into the extensible package. The source path and the (relative) target path inside the package can be specified.

- IP Library Metadata

  The following metadata of the created Extensible TLM package can be configured.

Display name	*Settings* > *General* > *IP Library* > *Library name*: This will be used as display name for showing the Extensible Model in the New TLM Project dialog.
Display category	*Settings* > *General* > *IP Library* > *Library display name*: If set, this will be used to group Extensible Models into an additional category node in the New TLM Project dialog.

- IP Library Redistribution

  The options that were already described above in the "IP Library Redistribution" on page 41 section for "Library Packaging" on page 40 also apply to the created Extensible Package.

4.3.3 Test Packaging

Test Packaging describes the process of exporting a TLM Model test package, that will enable building and executing the unit tests without requiring access to the original source project location. This is particularly useful when integrating with a build and regression system that requires the tests to be executed in a separate phase that is independent of the actual build phase.

Options for configuring a TLM Model test package are listed below.

- Package Content

  All required tool generated files will automatically be included in the test package. For additional contributions to the test package content, the following settings are available.

Settings - General > *Test Packaging*	This section can be used to add custom files into the test package. The source path and the (relative) target path inside the package can be specified.

< > < >

5.4. 4.4 VDK Building Blocks

The creation of Library Packages and Extensible Packages is supported for VDK Building Blocks. Details are explained in subsequent sections.

4.4.1 Library Packaging

Library Packaging describes the process of making a pre-built VDK Building Block available to downstream users. This is the regular flow for building and sharing VDK Building Block libraries with users that will instantiate and configure them in a VDK or subsystem context.

An option exists to also include an extensible version (that is, a source version, see "Extensible Packaging" on page 44) of the VDK Building Block inside the Library Package. This is useful for flows in which the receiving user need to modify and re-build the VDK Building Block locally.

Following are the options for configuring a VDK Building Block package.

- Package Content

  All required tool generated files will automatically be included in the library package. For additional contributions to the package content, the following settings are available.

Packaging > Packaging Actions > Library Packaging	This section can be used to add custom files into the library package. For external files, the source path and the (relative) target path inside the package can be specified.
Settings > IP Library > Include Extensible Package in Library Package	This flag controls whether the library package should also include an extensible version of the building block. Enabling this, allows downstream users to open, edit and re-build a the VDK building block inside their local workspace.

- IP Library Metadata

  The following metadata of the created IP library can be configured.

Model identifier in form of V/L/N/V (Vendor/Library/Name/Version)	Included under *Settings > Model Information*.
Hardware Category, Vendor, and Name	Included under *Settings > Hardware Information*.
Path variable name	The automatically generated path variable name can be overridden under *Settings > IP Library*.

- IP Library Redistribution

  Extensible VDKs can redistribute IP libraries that are used in the VDK design in form of an IP library package that contains all required IP libraries needed for being able to re-build the Extensible VDK. A VDK Building Block package can indicate whether it allows to be redistributed, and whether it has further dependents that should also be redistributed along with it.

  Following related settings are included in the *Packaging > Packaging Contributions > IP Library Redistribution* section of the *VDK Building Block* editor:

Allow redistribution of this library	Flag to control whether the VDK Building Block can be redistributed or not.
Dependent Libraries	List of dependent libraries that should also be redistributed if this VDK Building Block is included in an IP library package. Each entry in the list consists of: • *IP Library:* The name of the IP library. • *Type*: If the dependent IP library specifies multiple types, it is possible to select which one this VDK Building Block should depend on. By default, this field is empty which means that the entire dependent library (all types) should be included when redistributing. • *Target Type*: In case a library package also includes an extensible package, there can be cases where the pre-built building block and the extensible building block need to have different dependencies. The target type can be used to indicate that a dependency only applies to a sub-type of the created library package. Leaving it empty will result in the dependency to apply for the entire library package.
Automatically include all redistributable libraries used in this design	Flag to indicate that all redistributable IP libraries used in this design, should be added as a (packaging) dependency to the generated IP library. NOTE: Adding dependencies here, will not result in them to be copied into the VDK Subsystem library. Instead, those dependencies will be expressed as metadata in the created `*.iplib` file. This will allow, for example, Extensible VDK packaging to collect all dependencies recursively to create an IP library package that accompanies the Extensible VDK package (shown in a later Task).

Note	Adding dependent libraries to those settings, will not result in them to be copied into the create IP library but instead, the dependencies will be expressed as meta-data in the created IP library. This will allow, for example, Extensible VDK packaging to collect all dependencies recursively to create an IP library package that accompanies the Extensible VDK package.

4.4.2 Extensible Packaging

Extensible Packaging describes the process of making an Extensible VDK Building Block package available to downstream users. An Extensible VDK Building Block package contains a full source version of the design, allowing users to create, edit, and re-build local copies of it.

Extensible VDK Building Blocks will be available from the New VDK Project dialog under the *VDK Building Block* project type. Selecting an *Extensible VDK Building Block* in this dialog will result in a local project with a copy of the design files required to edit and re-build the building block. The modified building block from the local workspace project will effectively become a new building block that can again be packaged and redistributed like any regular VDK Building Block.

Following are options for configuring an Extensible VDK Building Block package.

- Package Content

  All required tool generated files will automatically be included in the extensible package. For additional contributions to the package content, the following settings are available.

Packaging > Packaging Actions > Extensible Packaging	This section can be used to add custom files into the library package. For external files, the source path and the (relative) target path inside the package can be specified.

- IP Library Redistribution

  The options that were already described above in the "IP Library Redistribution" on page 43 section for "Library Packaging" also apply to the created Extensible Package.

4.4.3 IP Library Packaging

IP Library Packaging describes the process of creating a package that contains the VDK Building Block and also all its dependent IP libraries (TLM models and/or VDK Building Blocks). This flow should only be used when not sharing an Extensible VDK (in this case section 4.5.2 applies) but only a VDK Building Block with a downstream user. This action will create an `*.ip.zip` file that contains:

- The current VDK Building Block's library package.
- All dependent libraries that are listed under *Packaging > Packaging Contributions > IP Library Redistribution > Dependent Libraries*.

< > < >

5.5. 4.5 Virtualizer Development Kits

The creation of Fixed and Extensible VDK packages is supported For Virtualizer Development Kits (VDKs). There is also an option to create a package without the actual VDK which is intended to be used to distribute optional sideband packages consisting of, for example., VP Configs, scripts, or IP libraries.

Multiple packaging configurations can be added to a VDK in the Packaging Tab of the VDK editor. Each packaging configuration will result in a dedicated *packaging* action becoming available. A *packaging* configuration can be of type Fixed VDK, Extensible VDK, or No VDK and can be configured as described below.

4.5.1 Fixed VDK Packaging

Fixed VDK packaging requires a Fixed VDK packaging configuration to be created. For more details, see Packaging VDK Designs of the *VDK Creator User Guide*.

- Package Content

  All required tool generated files will automatically be included in the package. Additional contributions to the package content can be configured as shown in the table below.

Additional Packaged Files	Can be added in the *Packaging > Fixed VDK* section.
VP Configs	Included VP Configs can be configured in the *Packaging > Packages > VP Configs* section.
Eclipse Projects	Included Eclipse Projects (for example, *PlatformTest*) can be configured in the *Packaging > Packages > Eclipse Projects* section.
Virtualizer Runtime Installer	The Package Type can be changed under *Packaging > Packages > Package Type* to *Fixed VDK with Virtualizer Runtime installer* to create a package consisting of the Fixed VDK itself and the Virtualizer Runtime installer.

- Fixed VDK Metadata

  The following metadata can be configured for Fixed VDKs in the *Packaging* > *Fixed VDK* section.

Fixed VDK Name	Name that is used for file and directory names when creating the Fixed VDK package. Defaults to the VDK name if left empty.
Display Name	Display name used on the *Welcome Screen* and in the New VDK Project dialog. Defaults to the Fixed VDK name if left empty.
Release Version	Fixed VDK specific release version. Should normally be left empty and will default to the VDK's model version which is configured in *Settings* > *General* > *Model Version*.
Hardware Name	Cannot be overridden here and will default to the VDK's hardware name which is configured in *Settings* > *General* > *Hardware Name*.
Documentation	The documentation path for the Fixed VDK.

4.5.2 Extensible VDK Packaging

Extensible VDK packaging requires an *Extensible VDK* packaging configuration to be created. For more details, see Packaging VDK Designs of the *VDK Creator User Guide*.

- Package Content

  All required tool generated files will automatically be included in the package. Additional contributions to the package content can be configured as shown in the table below.

Additional Packaged Files	Can be added in the *Packaging* > *Extensible VDK* section.
VP Configs	Included VP Configs can be configured in the *Packaging* > *Packages* > *VP Configs* section.
Eclipse Projects	Included Eclipse Projects (for example, *PlatformTest*) can be configured in the *Packaging* > *Packages* > *Eclipse Projects* section.

- IP Library Package

  Users of Extensible VDKs will require access to all used IP libraries to be able to build the VDK. The *Packaging* > *Packages* > *IP Libraries* section defines which IP libraries should be redistributed together with the Extensible VDK. This will result in a separate IP library package with the configured content to be created next to the Extensible VDK package.

  The following options exist to configure the IP library package content.

Included IP Libraries	This table can be used to explicitly add IP libraries to the created package. Each entry consists of • The IP library name. • The IP library type (for example, Pre-Built or Extensible). This is optional and the default is to include all types that are present. • The target path inside the created package. By default, libraries are copied into the top-level of the created package. NOTE: When adding an entry, dependent libraries will automatically be added as well and shown as a tree structure under the current entry.
Excluded IP Libraries	This can be used to remove unwanted libraries from the package content, for example, if they were added through dependencies of other libraries.
Automatically include all redistributable libraries used in this design	This flag can be set to indicate that all redistributable IP libraries used in this design should be included in the IP library package. All libraries that are found for this flag will be shown under a dedicated read-only entry in the table.

- Extensible VDK Metadata

  The following metadata can be configured for Extensible VDKs in the *Packaging* > *Extensible VDK* section.

Extensible VDK Name	Name that is used for file and directory names when creating the Extensible VDK package. Defaults to the VDK name if left empty.
Display Name	Display name used on the *Welcome Screen* and in the New VDK Project Dialog. Defaults to the Extensible VDK name if left empty.

Release Version	Extensible VDK specific release version. It should normally be left empty and will default to the VDK's model version which is configured in *Settings* > *General* > *Model Version*.
Documentation	The documentation path for the Fixed VDK.
Show on Welcome Screen	Flag to indicate that the created Extensible VDK should be shown on the *Welcome Screen*.
Show documentation after project creation	Flag to indicate whether the documentation tab should initially be shown when using the Extensible VDK.

4.5.3 No VDK Packaging

As explained above, Fixed and Extensible VDK packages can contain collaterals in form of VP Configs, Eclipse Projects, and/or IP libraries. It is possible to create a package that consists of only those collaterals and does not contain the actual (Fixed or Extensible) VDK design.

A packaging configuration of type *No VDK* can be created. It will support the same settings as listed above for Fixed/Extensible VDK packaging configurations. A corresponding packaging action will become available to create the *No VDK* package.

< > < >

5.6. 4.6 Fixed VDKs

A *Fixed VDK* project can be re-packaged into a new *Fixed VDK* package. It is possible to add/modify VP Configs and Eclipse Projects during re-packaging. However, it is not possible to modify the content of the actual Fixed VDK package.

4.6.1 Fixed VDK Re-Packaging

Except for the *Fixed VDK* package content settings, configuration of the Fixed VDK re-packaging is done by creating a packaging configuration of type *Fixed VDK* and is similar as described in "Fixed VDK Packaging" on page 45.

< > < >

6. Chapter 5
Virtualizer Studio Build System Integration

Virtualizer Studio Build System Integration (`vsmake`) is a command-line tool that is available next to the regular `vs` and `vssh` applications inside the Virtualizer installation. It is a headless version of Virtualizer Studio that provides the ability to integrate into a build and regression system to generate and test VDK and TLM projects. The capabilities have been designed to support the following aspects that are commonly encountered in build systems:

- Command-line support for building, packaging, and testing without having to maintain custom python scripts.
- Clean separation of build outputs towards an installation directory structure, avoiding using in-place builds and clearly separating the files that should be provided to an end user.
- Allowing for execution of regression tests to be run in a dedicated regression phase, after the build phase, and ensuring that the tests are executed against the installed output.

During interactive development of a TLM Model or VDK, the regular build action is typically used to build the project inside the local workspace. To build and export the actual IP library (in case of a TLM Model, TLM Building Block, or VDK Building Block) or the VDK package (in case of a VDK project), the packaging actions are supposed to be used. The packaging action creates and collects all required output files and installs them into a given output location. The packaging actions are also used by `vsmake` for integrating with a build system.

The following diagrams show a typical flow of the two main phases (build & test) that would be carried out by a build and regression system:

Figure 5-1 Typical Flow for a Separate Build Phase

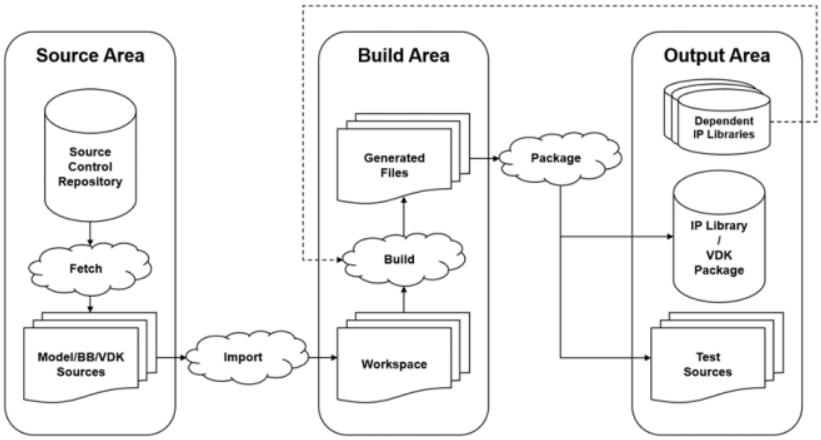


Figure 5-2 Typical Flow for a Separate Regression Testing Phase

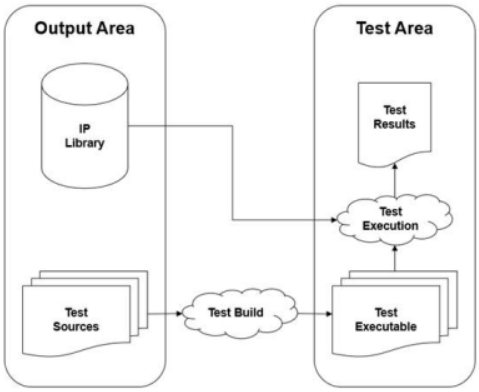
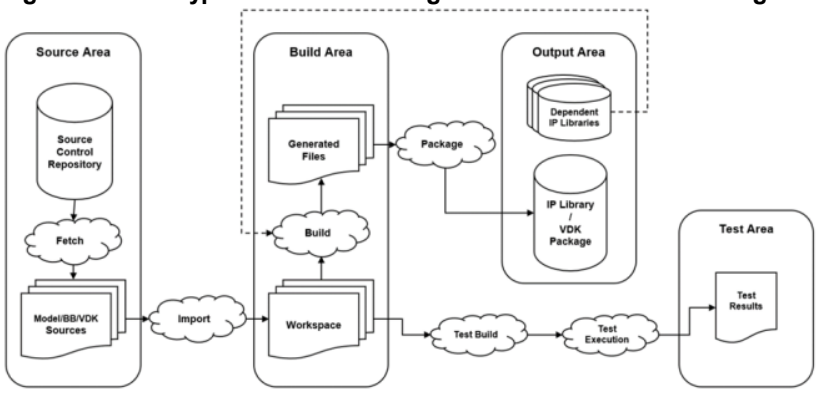


Figure 5-3 Typical Flow for a Single Combined Build and Regression Testing Phase



6.1. 5.1 Supported Projects and Actions

`vsmake` supports input projects of the following types:

- TLM Building Blocks
- TLM Models
- VDK Building Blocks
- VDKs
- Fixed VDKs

Typically, input projects will be present in the form of an existing TLM Creator or VDK Creator project (containing a `*.tlmc`, `*.vdksys`, or `*.vdkfxd` file). `vsmake` also supports flows where the input project either needs to be modified or even dynamically created by a Python script. Valid use flows for this can, for example, be creation of TLM Models by importing checked in SystemC sources or CSV tables.

`vsmake` supports the following high-level actions:

- Creating packages of types (for details, see "IP and VDK Packaging Flows" on page 35)
  - Library Package (for TLM Building Blocks, TLM Models, and VDK Building Blocks)
  - Extensible Package (for TLM Models and VDK Building Blocks)
  - Unit Test Package (for TLM Building Blocks and TLM Models)
  - Extensible VDKs (for VDKs)
  - Fixed VDKs (for VDKs and Fixed VDKs)
- Executing tests of types
  - Unit Tests (for TLM Building Blocks and TLM Models)
  - Virtualizer Studio BB Compliance Tests (for TLM Models and VDK Building Blocks). For details, see "Virtualizer Studio Building Block Compliance Testing" on page 59.

In general, two different use flows of separate or combined build and regression testing phases are supported as depicted in figures 5-1, 5-2, and 5-3 above and the basic invocation is as follows:

- Separate build and regression phases

  ```
  vsmake --package path/to/project
  ```

  This will run the packaging actions, but not execute any tests.
- Single combined build and regression phase

  ```
  vsmake --package path/to/project --test
  ```

  This will run the packaging actions and execute available tests.

For detailed description of the usage of those actions, see the next section Usage.

< > < >

6.2. 5.2 Usage

All intermediate files will be created in a dedicated build directory. This includes all log files of the build/test actions and the actual eclipse workspace. By default, it uses `./vsmake/build` which can be overridden using the `--workdir` argument:

```
vsmake --workdir path/to/builddir
```

The option `--rm` can be used to automatically delete this build directory after `vsmake` execution:

```
vsmake --workdir path/to/builddir --rm
```

`vsmake` will report status about the executed actions by printing to stdout. Depending on whether any error happens, `vsmake` will return with zero or non-zero exit code.

For details about potential errors, see the project name specific log files generated in the build directory.

The default structure of generated files looks the following:

Table 5-1 Generated Files

Generated Files	Description
`vsmake/output/artifact`	Specifies the default directory for IP/VDK packages.
`vsmake/output/test`	Specifies the default directory for test packages.
`vsmake/testresults/<IP_name>.log`	Specifies the default test output file with test results.
`vsmake/build/<IP_name>_package.log`	Specifies the default log file for packaging action.
`vsmake/build/<IP_name>_test.log`	Specifies the default log file for test execution.
`vsmake/build/<IP_name>_ws`	Specifies the default eclipse workspace location.

Note

- To clean the working directory after an invocation was successfull, use the option `--rm`.
- To ensure that the requested actions happen without any incremental optimizations, use the option `--pristine`.

5.2.1 Creating Packages

Creating a package from an input project is triggered by running

```
vsmake --package path/to/project
```

where `path/to/project` is the path to either an existing source project or a Python script that will create a project.

In case the argument is a path to an existing project, an additional Python script called `prebuild.py` will be executed before the actual build if it exists inside the project directory.

In case the arguments is a Python script, `vsmake` will execute the script and after that expect either a single project to be present in the workspace or the name of the project to be packaged to be found in a Python variable with name `vsMakeProjectName`.

Depending on the input project type, the following packages will be created by this invocation by default:

- TLM Building Blocks and TLM Models
  - Library and Unit Test Package
- VDK Building Blocks
  - Library Package
- VDKs and Fixed VDKs
  - One package for each packaging configuration the VDK project has

For TLM Building Blocks, TLM Models, and VDK Building Blocks, the packages to be created can also be overridden explicitly using the `--package_types` argument:

```
vsmake --package path/to/project --package_types <type1>[,<type2>,...]
```

Depending on the input project type, valid values for the package type are: `LIBRARY`, `EXTENSIBLE`, `UNITTEST`, `BLACKBOX`.

For VDKs and Fixed VDKs, a single packaging configuration must be selected to only generate a single package using the `--config` argument when the project contains more than a single packaging configuration:

```
vsmake --package path/to/vdk --config MyFixedVDKConfig
```

Note that for (re-)packaging of Fixed VDKs, access to the original template path that the Fixed VDK project was created from is needed. If this is not found in the default search paths, it can be injected using the `--dependencies-dir` argument:

```
vsmake --package path/to/fixed_vdk --config MyFixedVDKConfig \
  --dependencies-dir path/to/fixed_vdk_installation
```

The `--config` argument is also used to specify a non-default build configuration name when packaging a TLM Building Block or TLM Model:

```
vsmake --package path/to/tlm --config MyBuildConfig
```

To control the output location of each created package, the `--package-<type>-dir` option is used:

```
vsmake --package path/to/project \
  --package-LIBRARY-dir install/MyIP \
  --package-UNITTEST-dir testinstall/MyIP
```

In cases where one project is packaged and needs to be picked up from its install location by the build of a dependent project, the `--dependencies-dir` option can be used. This will add the given path as an additional path to the library manager of the project during the build:

```
vsmake --package path/to/MyModel --depencencies-dir install/MyBB
```

`vsmake` can override the debug/optimize/assertion settings of projects using the options:

- `--force-debug (on|off)`
- `--force-optimize (on|off)`
- `--force-assertions (on|off)`

It is also possible to set the number of parallel compile jobs `vsmake` uses using the option:

- `-j <number of jobs>`

Note

- Currently, only TLM Models and TLM Building Blocks use this parallelization to compile their specified source files in parallel.
- If multiple paths are required to package a project, the directories can be joined with a comma. For example:

  `--dependency-dirs dir1,dir2`

5.2.2 Running Tests

Tests can be executed from the same `vsmake` invocation that also does the packaging by simply adding the `--test` option without additional arguments:

```
vsmake --package path/to/project --test
```

Alternatively, in case a library package has been created from a previous invocation of `vsmake`, the tests can be executed by invoking:

```
vsmake --test path/to/library_package
```

For TLM Building Blocks and TLM Models, the location of the generated test package can be overridden using the `--package-UNITTEST-dir` argument if it is not found by `vsmake` under the default location:

```
vsmake --test path/to/library_package \
  --package-UNITTEST-dir path/to/unittest_package
```

Depending on the input project type, the following tests will be executed by this invocation by default:

- TLM Building Blocks
  - Unit Tests
- TLM Models
  - Unit Tests and VS BB Compliance Tests
- VDK Building Blocks
  - VS BB Compliance Tests

The tests to be executed can also be overridden explicitly using the `--test_types` argument:

```
vsmake --package path/to/project --test \
  --test_types <type1>[,<type2>,...]
```

Note	Unit Tests will always generate functional coverage reports in the file `Tests/Unittests/functional_coverage.html` under the unittest package. For more information, see "Functional Coverage" of the *TLM Creator User Guide* and "Functional Coverage" of the *SystemC Modeling Library Reference Manual*. Automated line coverage generation is currently unsupported.

or

```
vsmake --test path/to/library_package \
  --test_types <type1>[,<type2>,...]
```

Depending on the input project type, valid values for the test type are: `UNITTEST` and `COMPLIANCETEST`.

By default, test results will be stored in `./vsmake/testresults` which can be overridden using the `--testdir` argument:

```
vsmake --package path/to/project --test -testdir path/to/testresults
```

or

```
vsmake --test path/to/library_package --testdir path/to/testresults
```

< > < >

6.3. 5.3 Licensing Requirements

Both the packaging and testing phases will consume licenses. Executing multiple builds or regression test runs in parallel will consume multiple licenses.

The build phase will require a license that supports TLM model or VDK authoring, whereas the regression phase can use a lower license that supports running a simulation. If the IP being developed and tested is licensed, this will also be consumed when running the regression tests.

< > < >

6.4. 5.4 Quick References

This section contains example invocations of `vsmake` for different use cases. For improved readability, all examples assume those variables to be set up front:

```
export SOURCE=/path/to/sourcedir
export BUILD=/path/to/builddir
export TEST=/path/to/testdir
export INSTALL=/path/to/installdir
```

5.4.1 TLM Building Block Projects

Create Library and Unit Test Packages and Run Unit Tests:

```
vsmake --package $SOURCE/MyBB \
  --test \
  --workdir $BUILD \
  --testdir $TEST \
  --package-LIBRARY-dir $INSTALL/IP/MyBB \
  --package-UNITTEST-dir $INSTALL/tests/MyBB
```

Create Library and Unit Test Packages:

```
vsmake --package $SOURCE/MyBB \
  --workdir $BUILD \
  --package-LIBRARY-dir $INSTALL/IP/MyBB \
  --package-UNITTEST-dir $INSTALL/tests/MyBB
```

Run Unit Tests (from existing packages):

```
vsmake --test $INSTALL/IP/MyBB \
  --workdir $BUILD \
  --testdir $TEST \
  --package-UNITTEST-dir $INSTALL/tests/MyBB
```

Create Library Package Only:

```
vsmake --package $SOURCE/MyBB \
  --workdir $BUILD \
  --package_types LIBRARY \
  --package-LIBRARY-dir $INSTALL/IP/MyBB
```

5.4.2 TLM Model Projects

Create Library and Unit Test Packages, Run Unit Tests and Compliance Tests:

```
vsmake --package $SOURCE/MyModel \
  --test \
  --workdir $BUILD \
  --testdir $TEST \
  --package-LIBRARY-dir $INSTALL/IP/MyModel \
  --package-UNITTEST-dir $INSTALL/tests/MyModel
```

Create Library and Unit Test Packages, Run Unit Tests Only:

```
vsmake --package $SOURCE/MyModel \
  --test \
  --test_types UNITTEST \
  --workdir $BUILD \
  --testdir $TEST \
  --package-LIBRARY-dir $INSTALL/IP/MyModel \
  --package-UNITTEST-dir $INSTALL/tests/MyModel
```

Create Library and Unit Test Packages:

```
vsmake --package $SOURCE/MyModel \
  --workdir $BUILD \
  --package-LIBRARY-dir $INSTALL/IP/MyModel \
  --package-UNITTEST-dir $INSTALL/tests/MyModel
```

Run Unit Tests and Compliance Tests (from existing packages):

```
vsmake --test $INSTALL/IP/MyModel \
  --workdir $BUILD \
  --testdir $TEST \
  --package-UNITTEST-dir $INSTALL/tests/MyModel
```

Run Unit Tests Only (from existing packages):

```
vsmake --test $INSTALL/IP/MyModel \
  --test_types UNITTEST \
  --workdir $BUILD \
  --testdir $TEST \
  --package-UNITTEST-dir $INSTALL/tests/MyModel
```

Run Compliance Tests Only (from existing packages):

```
vsmake --test $INSTALL/IP/MyModel \
  --test_types COMPLIANCETEST \
  --workdir $BUILD \
  --testdir $TEST
```

Create Library Package Only:

```
vsmake --package $SOURCE/MyModel \
  --workdir $BUILD \
  --package_types LIBRARY \
  --package-LIBRARY-dir $INSTALL/IP/MyModel
```

Create Extensible Package Only:

```
vsmake --package $SOURCE/MyModel \
  --workdir $BUILD \
  --package_types EXTENSIBLE \
  --package-EXTENSIBLE-dir $INSTALL/IP/MyModel_ext
```

5.4.3 VDK Building Block Projects

Create Library Package and Run Compliance Tests:

```
vsmake --package $SOURCE/MyVdkBB \
  --test \
  --workdir $BUILD \
  --testdir $TEST \
  --package-LIBRARY-dir $INSTALL/IP/MyVdkBB
```

Create Library Package:

```
vsmake --package $SOURCE/MyVdkBB \
  --workdir $BUILD \
  --package-LIBRARY-dir $INSTALL/IP/MyVdkBB
```

Create Library Package with Blackbox Subsystem:

```
vsmake --package $SOURCE/MyVdkBB \
  --workdir $BUILD \
  --package_types BLACKBOX \
  --package-BLACKBOX-dir $INSTALL/IP/MyVdkBB_bb
```

Run Compliance Tests (from existing packages):

```
vsmake --test $INSTALL/IP/MyVdkBB \
  --workdir $BUILD \
  --testdir $TEST
```

Create Extensible Package Only:

```
vsmake --package $SOURCE/MyVdkBB \
  --workdir $BUILD \
  --package_types EXTENSIBLE \
  --package-EXTENSIBLE-dir $INSTALL/IP/MyVdkBB_ext
```

5.4.4 VDK Projects

Create VDK Package from Single Existing Packaging Configuration:

```
vsmake --package $SOURCE/MyVdk \
  --workdir $BUILD \
  --package-LIBRARY-dir $INSTALL/VDK
```

Create Fixed VDK Package from Packaging Configuration "MyFixedVDK":

```
vsmake --package $SOURCE/MyVdk \
  --config MyFixedVdk \
  --workdir $BUILD \
  --package-LIBRARY-dir $INSTALL/VDK
```

Create Extensible VDK Package from Packaging Configuration "MyExtensibleVDK":

```
vsmake --package $SOURCE/MyVdk \
  --config MyExtensibleVdk \
  --workdir $BUILD \
  --package-LIBRARY-dir $INSTALL/VDK
```

5.4.5 Fixed VDK Projects

Create VDK Package from Existing Single Packaging Configuration:

```
vsmake --package $SOURCE/MyFixedVdk \
  --workdir $BUILD \
  --dependencies-dir $INSTALL/VDK \
  --package-LIBRARY-dir $INSTALL/VDK_repackaged
```

Create Fixed VDK Package from Packaging Configuration "MyFixedVDKConfig":

```
vsmake --package $SOURCE/MyFixedVdk \
  --config MyFixedVdkConfig \
  --workdir $BUILD \
  --dependencies-dir $INSTALL/VDK \
  --package-LIBRARY-dir $INSTALL/VDK_repackaged
```

5.4.6 Python-Based Projects

TLM Model: Create Library and Unit Test Packages, Run Unit Tests and Compliance Tests:

`create.py` needs to create a single TLM Model project in the workspace or set `vsMakeProjectName = "ModelName"` to indicate that `ModelName` is the project to be packaged if multiple projects are created.

```
vsmake --package $SOURCE/MyDynamicModel/create.py \
  --test \
  --workdir $BUILD \
  --testdir $TEST \
  --package-LIBRARY-dir $INSTALL/IP/MyDynamicModel \
  --package-UNITTEST-dir $INSTALL/tests/
```

Example skeleton for a script running a CSV import:

```
# Step 1: create a new model
model = tlmcreator.new_model('MyModel')

# Step 2: import from CSV file
ImportCSVFromRulesFile("MyRulesFile.rls", model)

# Step 3: update meta-data and build settings as needed
# ...

# Step 4: save the model
model.save()
```

< > < >

7. Chapter 6
Virtualizer Studio Building Block Compliance Testing

Virtualizer Studio provides a feature to run a set of tests against an IP library and report the compatibility of the building blocks for use within Virtualizer Studio. The primary goal of these tests is to provide confidence that building blocks will operate correctly in different circumstances. They can be run against an entire IP library, or against a single individual building block within an IP library, and will generate a text report which can be parsed manually, or automatically by a regression test infrastructure.

This chapter describes:

- Test Scenarios
- Running the Tests
- Test Configuration Options
- Test Results

< > < >

7.1. 6.1 Test Scenarios

A predefined set of test scenarios will be run against each building block. Each test scenario will consist of a series of finer granular tests. Certain tests may not be applicable to all building blocks, and can be skipped where appropriate.

This section includes:

- Run a VDK Containing the Building Block
- Package a VDK Containing the Building Block
- Creating a Subsystem Containing the Building Block

- Runtime Disabling the Building Block
- Presence of Collateral Files in Building Block Library

6.1.1 Run a VDK Containing the Building Block

This test scenario will instantiate the building block under test (`BBUT`) into a VDK design, and will ensure that the VDK can be built and launched successfully.

The following tests are executed as part of this test scenario:

Table 6-1 Tests Executed in the Test Scenario

Test ID	Description
`Assemble_BBUT_VDK`	Instantiates an instance of the building block under test into an empty VDK design.
`Build_BBUT_VDK`	Builds the VDK containing the building block under test.
`Launch_BBUT_VDK`	Launches the VDK containing the building block under test.
`Build_Fallback_BBUT_VDK`	Builds the VDK containing a non-prebuilt configuration of the building block under test.

For building blocks which have template arguments, an explicit test is executed to configure the instance with a set of template arguments known not to be pre-built into the DLL, and therefore requiring a fallback build of the building block. The precise template arguments are passed into the test infrastructure using the configuration file.

6.1.2 Package a VDK Containing the Building Block

This test scenario will create a *Fixed VDK* from the VDK containing the instance of the building block under test, and will ensure that this *Fixed VDK* can be launched successfully.

The following tests are executed as part of this test scenario:

Table 6-2 Tests Executed in the Test Scenario

Test ID	Description
`Package_Fixed_BBUT_VDK`	Packages the VDK containing the building block under test into a *Fixed VDK* package.
`Launch_Fixed_BBUT_VDK`	Extracts and launches the *Fixed VDK* containing the building block under test.

6.1.3 Creating a Subsystem Containing the Building Block

This test scenario will create a VDK Building Block (`SSBB`) which contains an instance of the building block under test. It will subsequently test the use of the subsystem in a *VDK* and a *Fixed VDK*, similar to the scenarios executed directly against the building block under test.

The following tests are executed as part of this test scenario:

Table 6-3 Tests Executed in the Test Scenario

Test ID	Description
`Assemble_BBUT_SSBB`	Instantiates an instance of the building block under test into an empty subsystem building block design and externalizes the interfaces.
`Build_BBUT_SSBB`	Builds the subsystem design containing the building block under test.
`Assemble_SSBB_VDK`	Instantiates an instance of the subsystem building block into an empty VDK design.
`Build_SSBB_VDK`	Builds the VDK containing the subsystem building block.
`Launch_SSBB_VDK`	Launches the VDK containing the subsystem building block.

`Package_Fixed_SSBB_VDK`	Packages the VDK containing the subsystem building block into a *Fixed VDK* package.
`Launch_Fixed_SSBB_VDK`	Extracts and launches the *Fixed VDK* containing the subsystem building block.

6.1.4 Runtime Disabling the Building Block

This test scenario will test the runtime disable capability of this building block, by ensuring that a VDK containing an instance of the building block (and a subsystem containing this building block) can be successfully launched when those instances are disabled.

The following tests are executed as part of this test scenario:

Table 6-4 Tests Executed in the Test Scenario

Test ID	Description
`Runtime_Disable_BBUT`	Launches the VDK with an instance of the building block under test disabled.
`Runtime_Disable_SSBB`	Launches the VDK with an instance of the subsystem building block disabled.

6.1.5 Presence of Collateral Files in Building Block Library

This test scenario will ensure that the expected collateral files related to this building block are present in the IP library.

The following tests are executed as part of this test scenario:

Table 6-5 Tests Executed in the Test Scenario

Test ID	Description
`Presence_Documentation`	Checks that a documentation file has been specified for this building block, and that the file is present in the IP library.

< > < >

7.2. 6.2 Running the Tests

The tests are run from the command line, by invoking Virtualizer Studio with specific command-line arguments. The tests can be run using either `vs` or `vssh`.

This section includes:

- Command-Line Arguments
- Running Multiple Tests

6.2.1 Command-Line Arguments

The following command-line arguments are relevant to running the Virtualizer Studio Building Block Compliance Tests.

Table 6-6 Command-Line Arguments

Flag	Description	Comments
`--bb_test_library`	Triggers a run of the Virtualizer Studio Building Block Compliance Tests. Specifies the IP library containing the building block(s) to be tested.	Followed by the file path to the library XML file.
`--bb_test_models`	Specifies the building block(s) to be tested when running the building block compliance tests. If this argument is not specified, the default value `all` is used.	Followed by either `all` or a comma-separated list of model specifiers. Model specifiers are listed in the format `Vendor/Library/Name[/Version]`.

`--bb_test_config`	Specifies the input file containing the configuration options when running the building block compliance tests. If this argument is not specified, the default configuration options are used.	Followed by the file path.
`--bb_test_results`	Specifies the output file containing the test results when running the building block compliance tests. If this argument is not specified, the output is directed into the regular logfile or output stream.	Followed by the file path.

The following command line shows an example of how to execute Virtualizer Studio Building Block Compliance Tests against a locally building block called `MyModel`:

```
vssh \
  -d ./MyModelBBTestWS \
  --bb_test_library ../MyModel/lib/MyModel.xml \
  --bb_test_models all \
  --bb_test_config ../MyModel/tests/MyModelBBTestConfig.txt \
  --bb_test_results ./MyModelBBTestResults.txt \
  --logfile ./MyModelBBTestOutput.txt
```

6.2.2 Running Multiple Tests

To run building block compliance tests on multiple building blocks in different IP libraries, `vssh` must be invoked multiple times.

To run building block compliance tests on multiple building blocks in the same IP library, the `bb_test_models` command-line argument can be used to specify one of the following:

- Run the building block compliance tests against one specified building block in this IP library.
- Run the building block compliance tests against a list of specified building blocks in this IP library.
- Run the building block compliance tests against all building blocks in this IP library.

Note	When running building block compliance tests against multiple building blocks in the same invocation of `vssh`, the tests are run sequentially.

< > < >

7.3. 6.3 Test Configuration Options

Some tests will require specific configuration options to be passed to the test infrastructure. Those options are passed in the form of a configuration file, which can be specified in the `bb_test_config` command-line argument.

This section includes:

- Fallback Template Arguments
- Skipping Tests
- Parameter Overrides
- Additional Libraries

6.3.1 Fallback Template Arguments

For any building block that contains template arguments, the test should be of one specific valid combination of template arguments for which there is no pre-built variant of the code inside the DLL. This will test that the *fallback build* mechanism of the model is successful. In such cases, one valid combination of template arguments to be tested can be specified in this configuration file.

An example of specifying a valid set of template arguments for a building block is:

```
{
"fallback_template_arguments":
    { "indexed_value_muxer": { "DT": "unsigend long" } }
}
```

For building blocks with no template arguments, this test will be skipped automatically, and nothing needs to be specified.

For building blocks with all valid combinations of template arguments pre-built into the DLL, this test should be explicitly skipped.

6.3.2 Skipping Tests

Tests can be skipped if there is a valid reason why this building block should not support a certain feature. They will be explicitly listed as being skipped in the test results, along with the reason specified in this configuration file.

An example of specifying that certain tests should be skipped is:

```
{
"skipped_tests":
    { "Build_Fallback_BBUT_VDK": { "indexed_value_muxer": "Pre-built for all values" } }
}
```

6.3.3 Parameter Overrides

A building block should be able to be instantiated successfully using its default parameter values. However, there can be cases where a valid default cannot be selected by the building block creator, and therefore needs to be set when the building block is instantiated. A typical example of this is a path to the required runtime file, which is not part of the building block library. In such cases, parameter overrides can be specified in this configuration file.

An example of specifying parameter overrides for a certain building block is:

```
{
"parameter_overrides":
    { "indexed_value_muxer": { "DT": "bool" } }
}
```

6.3.4 Additional Libraries

Some building blocks require additional IP libraries to be available to the library manager in order to run the test successfully. A typical example of this would be if the building block contained interfaces of a custom plane or protocol, which needs to be known to the tool, along with registered stubs, forwarders and so on. In such cases, additional libraries can be specified in this configuration file.

An example of specifying additional libraries for a certain building block is:

```
{
"extra_libraries":
    { "indexed_value_muxer": [ "/custom/path/stubs.vdklib", "/another/path/foo.xml" ] }
}
```

< > < >

7.4. 6.4 Test Results

The tests will generate a results file, as specified by the `bb_test_results` command-line argument. This file is in the form of a text file, and can be automatically parsed by a regression test infrastructure.

The following information can easily be extracted from this results file:

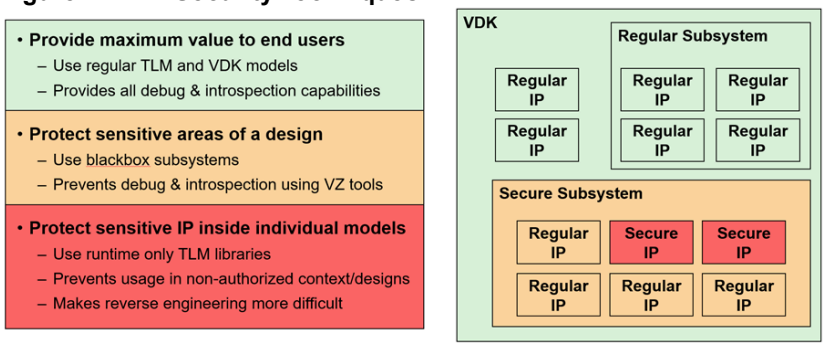
- General information:
  - Timestamp of the start of the compliance test run.
  - Model specifier of building block under test.
  - Compiler for which the building block is being tested.
  - Environment information (OS, Architecture).
- Information for each test feature being executed:
  - Name of test feature.
  - Timestamp of start of test.
  - Timestamp of end of test.
  - Status of test (`SUCCESS`, `FAIL` or `SKIP`).
  - Duration of test.
  - Reason for skipping test.

8. Appendix A
Enhanced Security for Sensitive IP and VDKs

One of the key strengths of a VDK is the ability to provide the software developer with useful information about the state of both the simulated hardware, and the software running on the simulated hardware. Whilst maximizing the visibility and debug capabilities is normally beneficial to the user of a VDK, there are some use cases where a VDK provider may have a stronger requirement to restrict access and visibility into more sensitive areas of the design.

Virtualizer provides some security techniques that enable VDK providers to balance the security requirements against the capabilities available to the end users. The following figure provides an overview of the techniques that can be used, depending on the security requirements.

Figure A-1 Security Techniques



This appendix describes:

- Provide Maximum Value to End Users
- Protect Sensitive Areas of a Design
- Protect Sensitive IP Inside Individual Models
- Details of Restrictions for Each User

8.1. A.1 Provide Maximum Value to End Users

In cases where there are no particular security concerns, regular use of TLM and VDK models will result in the end user having maximum introspection, debug, and analysis capabilities on both the hardware and the embedded software. This is the regular usage of Virtualizer Studio when no specific security techniques are applied.

8.2. A.2 Protect Sensitive Areas of a Design

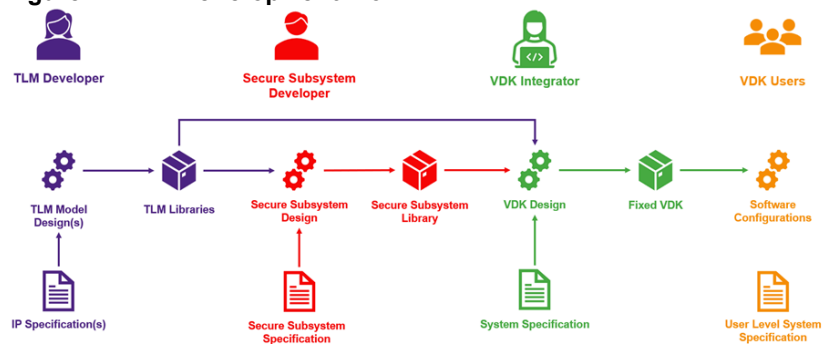
The blackbox subsystem technique is the recommended approach to protect visibility into the contents and connectivity of a sensitive section of a design, as well as preventing runtime introspection and interaction with any of the models and prevent debugging any embedded software running on cores inside this part of the design.

This technique involves one privileged user with access to the specifications of the sensitive part of the design, designing this part of the design in a separate subsystem, controlling which interfaces and parameters are exposed, and building this subsystem using a special build mechanism. The output of this build is a blackbox subsystem library that can be provided to a VDK integrator (along with TLM libraries for all IP inside the subsystem), who can integrate this sensitive subsystem into the larger VDK design, without having any insight into the contents or the connectivity of the sensitive part of the design. This user is solely responsible for instantiating this subsystem as a blackbox and connecting its external interfaces to the rest of the system.

The end user of the VDK will not have any visibility, introspection, or debug capabilities, but any embedded software running on cores inside this subsystem will still run securely, without the ability to connect an embedded software debugger.

The following diagram describes the different users in this development flow, and the packages that they each provide to the next user.

Figure A-2 Development Flow



8.3. A.3 Protect Sensitive IP Inside Individual Models

Sensitive IP or algorithms inside individual TLM models can be further protected by providing runtime only TLM libraries to the VDK integrator. The goals of this technique are to prevent usage of the TLM model in any non-authorized context, and to make reverse engineering of the model implementation more difficult.

This technique is typically used in conjunction with a blackbox subsystem and would be applied to any TLM models of IP inside such a secure subsystem which are considered highly sensitive or contain any sensitive algorithmic functionality.

The secure subsystem developer will need to be provided with a regular version of the TLM library, to correctly assemble and build the secure subsystem. However, any subsequent users, including the VDK integrator would be provided with a runtime only version of the TLM library, which provides a minimal set of functionality to build and run the VDK, but actively restricts the use of this TLM model to only be usable within the context of the secure subsystem.

The main limitations of the runtime only TLM library are that the TLM model cannot be instantiated in any other design, preventing any malicious misuse by the VDK integrator, and many of the symbols are removed from the DLL, making it more difficult for any malicious user of the VDK to reverse engineer the model's implementation.

The following table summarizes the key differences between the different types of IP libraries.

Table A-1 IP Libaries Type Differences

	Regular TLM Library	Extensible TLM Library	Runtime Only TLM Library	Fixed VDK Package
Model DLL(s), capable of simulating the model	Yes	Yes	Yes	Yes
Ability for the Virtualizer Studio library manager to locate the TLM library (*.pctlib file)	Yes	Yes	Yes	No
Ability for the TLM model to be instantiated in any new design (*.xml file)	Yes	Yes	No	No
Symbolic information present in model DLL	Yes	Yes	No	No[1]
Model source code, and ability to rebuild the model DLL	No	Yes	No	No
Model documentation	Yes	Yes	No	No

[1] Presence of symbols is dependent on the type of TLM library available to the VDK integrator.

8.4. A.4 Details of Restrictions for Each User

The following table provides more detail regarding the impact on each user when using blackbox subsystems and runtime only TLM libraries.

Table A-2 User Restrictions

	Secure Subsystem Developer	**VDK Integrator**	**VDK User**
Minimum TLM library type for models inside the secure subsystem	Regular TLM Library	Runtime Only TLM Library	n/a (files provided by Fixed VDK packaging)
Ability to perform full source code debug on a TLM model inside the secure subsystem	No[1]	No[1]	No[1]
Ability to view implementation specific symbols on the DLL of the TLM model	Yes	No[2]	No[2]
Ability to instantiate the same TLM model in any other design	Yes	No[2]	No
Ability to view the list of peripherals inside the secure subsystem	Yes	No	No
Ability to view the connectivity (memory map, interrupts, signals and so on.) of the peripherals inside the secure subsystem	Yes	No	No
Ability to modify the parameters (including software image) of any peripherals inside the secure subsystem, not explicitly externalized to the subsystem boundary	Yes	No	No
Ability to introspect the internals of the peripherals (that is, registers, signals, and so on.) at simulation runtime, using Virtualizer Studio's debug features	n/a[3]	No	No
Ability to send commands to the model at runtime, either interactively or from a VP Script	n/a[3]	No	No
Ability to interact with the model using SimProbe scripts	n/a[3]	No	No
Ability to view the state of any embedded software running on a core inside the secure subsystem, or connect an embedded software debugger	n/a[3]	No	No

[1] Would be available if the user was explicitly provided with an Extensible TLM Library instead.

[2] Assumes the VDK integrator is provided with a Runtime Only TLM Library, as opposed to a Regular (or Extensible) TLM Library.

[3] Subsystem author cannot simulate the subsystem in the full VDK context, but could access those features on the same TLM model, when used within a different context.